

Basic Numpy Operations:

- import pandas as pd
- 1) `arr = np.array([10, 20, 30, 40])` # Printing an array
`Print(arr)`
- 2) `arr_2 = np.array([[1, 2, 3], [4, 5, 6]])`
`Print(arr_2)`
`print(arr_2.shape)` # Printing shape of an array
- 3) `a = np.zeros((2, 3))` # print a matrix having all values to 0
`print(a)`
- 4) `a = np.ones((2, 3))` # Print a matrix having all values to 1.
`Print(a)`
- 5) `a = np.arange(1, 10, 2)` # Creates a numpy array with a range of values.
- 6) `arr = np.array([10, 20, 30, 40, 50])`
`print(arr[0])` # Array indexing & slicing
`print(arr[1:4])`
`print(arr[-1])`
- 7) `a = np.array([[1, 2], [3, 4]])` # Addition,
`b = np.array([[4, 3], [2, 1]])` Subtraction,
Multiplication,
Transpose
`print(a + 1)`
`print(b - 2)`
`print(a + b)`
`print(a @ b)`
`print(a.T)`


```
8) print(np.sum(arr))  
   print(np.sqrt(arr))  
   print(np.mean(arr))  
   print(np.max(arr))  
   print(np.min(arr))  
   print(np.std(arr))
```

basic mathematical
Operations.

```
9) arr = np.array([1, 2, 3, 4, 5, 6])  
   np.reshape(arr, (-1, 2)) # reshaping the array
```

```
10) a = np.array([[5, 6], [7, 8]])  
     res = a.flatten()  
     print(res)
```

* Basic Operations of Pandas :-

```
1) import pandas as pd  
   s = pd.Series({'Amit': 85, 'Riya': 90, 'Karen': 78})  
   print(s)  
   print(s[1])  
   print(s['Riya'])  
   print(s.max())  
   print(s.min())  
   print(s.mean())  
   print(s.std())
```


2) Data = { 'Name': ['Amit', 'Riya', 'Karan'],
 'Marks': [85, 90, 78],
 'City': ['Delhi', 'Mumbai', 'Pune'] }

df = pd.DataFrame(data, index = ['R1', 'R2', 'R3'])

Print(df)

print(df.columns)

print(df.index)

print(df.head())

print(df.tail())

print(df.info())

print(df.loc[i])

print(df.iloc[i])

print(df.loc[0:2])

print(df.iloc[0:2])

3)

	Name	Marks
a	Karan	10
b	Aditi	20
c	Harshit	30
d	Neha	25

new_rows = pd.DataFrame({ 'Name': ['Vijay', 'Rohit'],
 'Marks': [30, 40] })

pd.concat([df, new_rows], axis=0)

pd.concat([df, new_rows], ignore_index=True)

df.drop(1, axis=0)

df.drop([1, 3], axis=0)

df.sort_values(by='Marks')

df.sort_values(by='Marks', ascending=False)

df.rename(columns: {'Marks': 'score'})

df.isnull()

df.dropna()

4) Graph:

```
import numpy as np
```

```
import seaborn as sns
```

```
x = np.array([5, 10, 15, 20, 25, 30])
```

```
y = np.array([10, 20, 30, 40, 50, 60])
```

```
import matplotlib.pyplot as plt
```

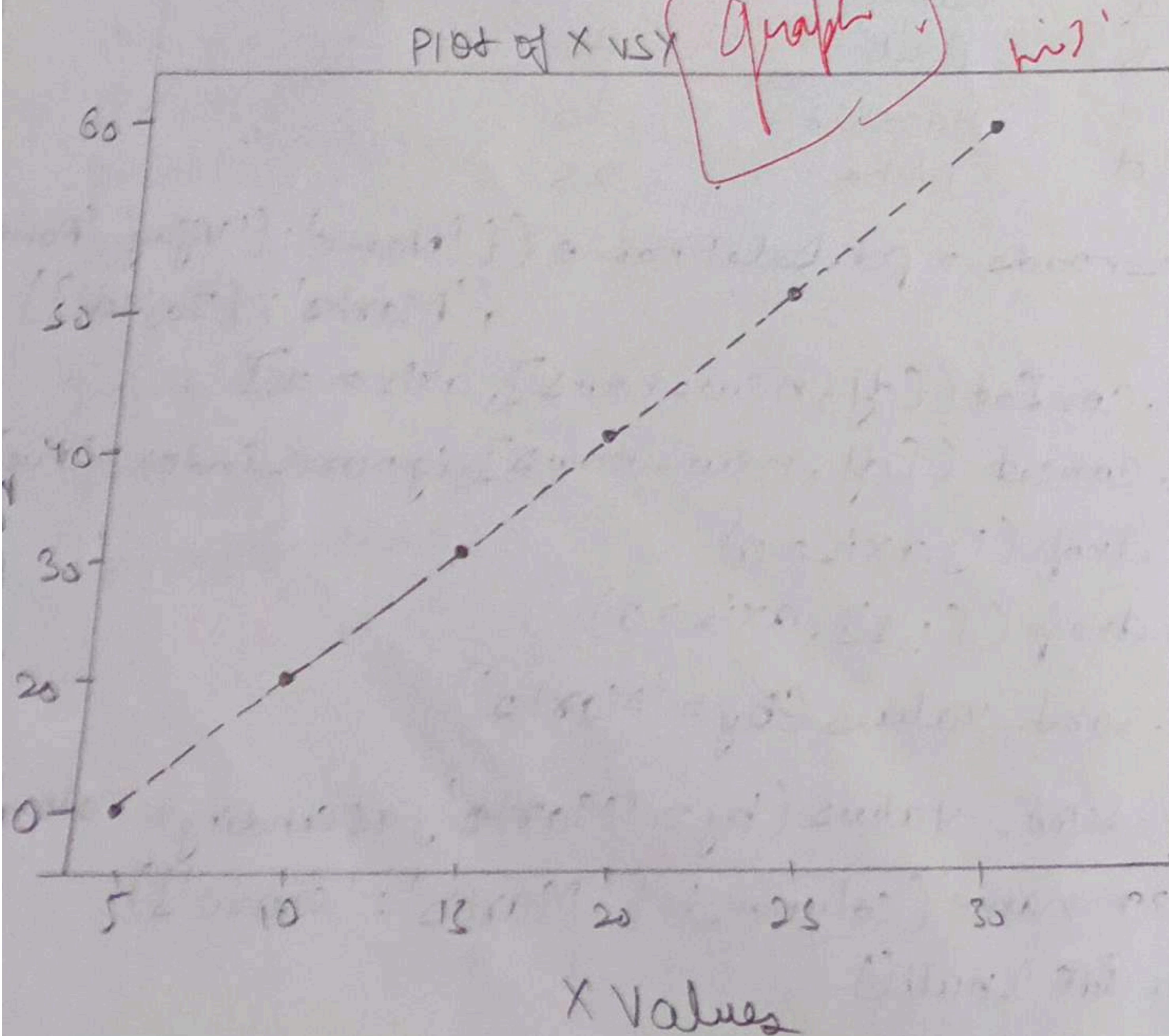
```
plt.figure(figsize=(6,4))
```

```
plt.plot(x, y, color='blue', marker='o', linestyle='--')
```

```
plt.title("line graph of x vs y")
```

```
plt.xlabel("X Values")
```

```
plt.ylabel("Y Values")
```



16/1/26

Create a dataset representing the medical history of patients and then analyze it using Pandas and Matplotlib.

Ans) Medical-History.csv

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
df = pd.read_csv("@file path")
df
```

→ ~~filling missing values~~

~~df["Age"] = df["Age"].fillna(df["Age"].median())~~

~~df~~

→ df.head()

Output:- Shows the 5 first Rows of Dataset by default.

	age	sex	bmi	children	Smoker	region	charges
0	19	female	27.900	0	Yes	Southwest	16884.6
1	18	male	33.770	1	no	Southwest	24658.7
2	28	male	33.000	3	no	Northwest	4449.4
3	32	male	22.705	0	no	Southwest	21984.4
4	33	male	28.880	0	no	Eastwest	3866.8

→ df.tail()

* Shows the ~~Rows~~ 5 last Rows by default.

→ df.sample(5)

* gives random rows from Datasets.

→ df.info()

* Provides summary of the datasets, includes column names, data types, and non-null values.

#	Column	Non-Null Count	Dtype
0	age	99 non-null	int64
1	sex	99 non-null	object
2	bmi	99 non-null	float64
3	children	99 non-null	int64
4	smoker	99 non-null	object
5	region	99 non-null	object
6	charges	99 non-null	float64
			object(3)

→ df.shape

* Return the number of rows and columns in the dataset.

(11, 5)

→ df.describe

* Return Statistical Summary

	age	bmi	children	charge
count	99.0	99.0	99.0	99.0
mean	38.8	30.9	1.080	14575.9
std	14.9	5.6	1.242	11375.01
min	18.0	17.3	0.00	4295.47
25%	26.5	26.7	0.00	10602.38
50%	37.0	31.1	1.00	20867.71
75%	55.0	37.5	2.00	51194.55
max	64.0	42.13	5.00	

Create a dataset of sales having columns id, customer name, name of product, cost of product, quantity, discount.

→ `df.unique()`

→ Filling null values :-

`df.isnull().sum()`

`df["age"].fillna(df["age"].mean(), inplace=True)`

`df["bmi"].fillna(df["bmi"].mean(), inplace=True)`

`plt.figure()`

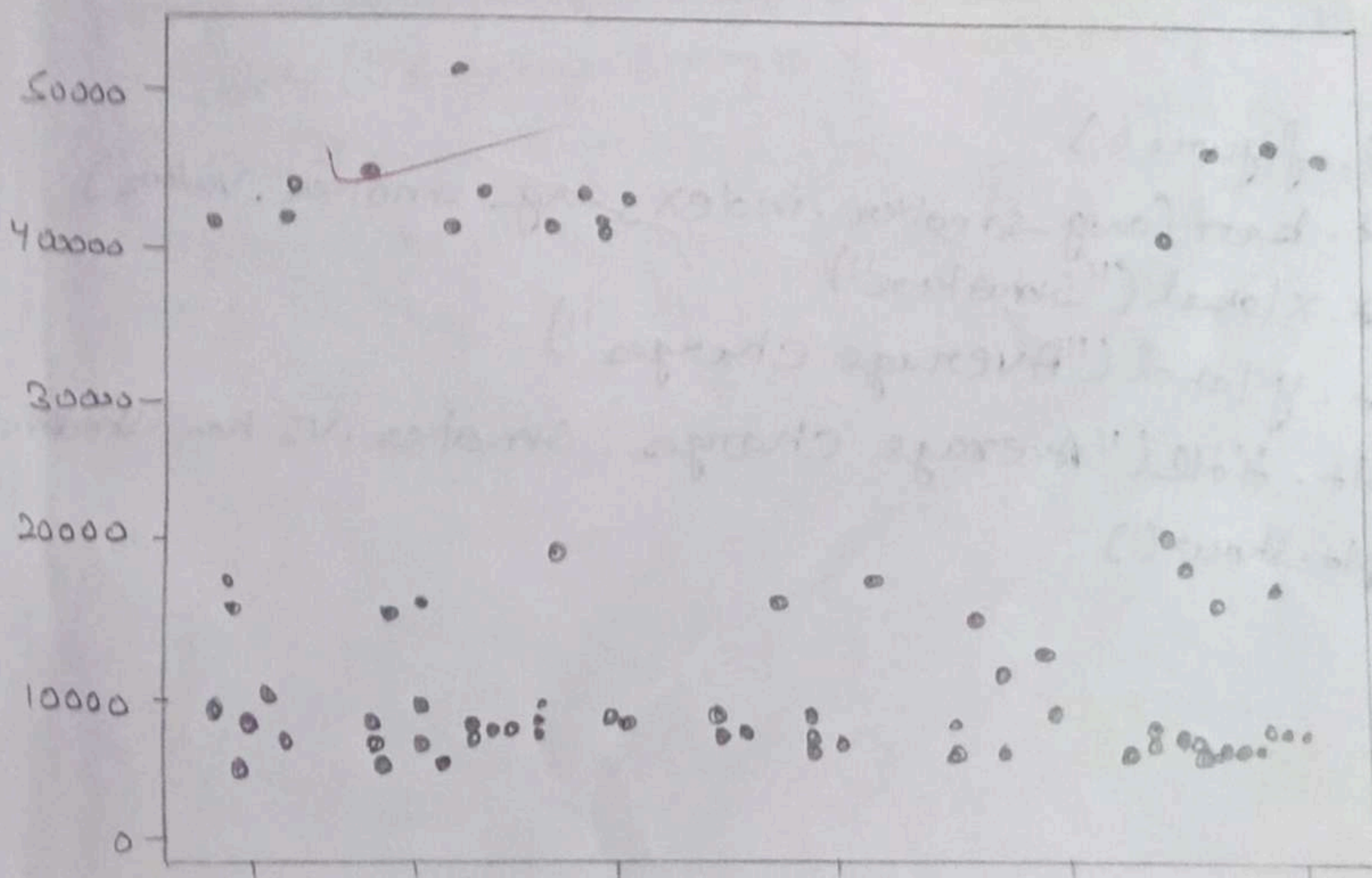
`plt.scatter(df["age"], df["charges"])`

`plt.xlabel("Age")`

`plt.ylabel("Charges")`

`plt.title("Age VS Insurance Charges")`

`plt.show()`

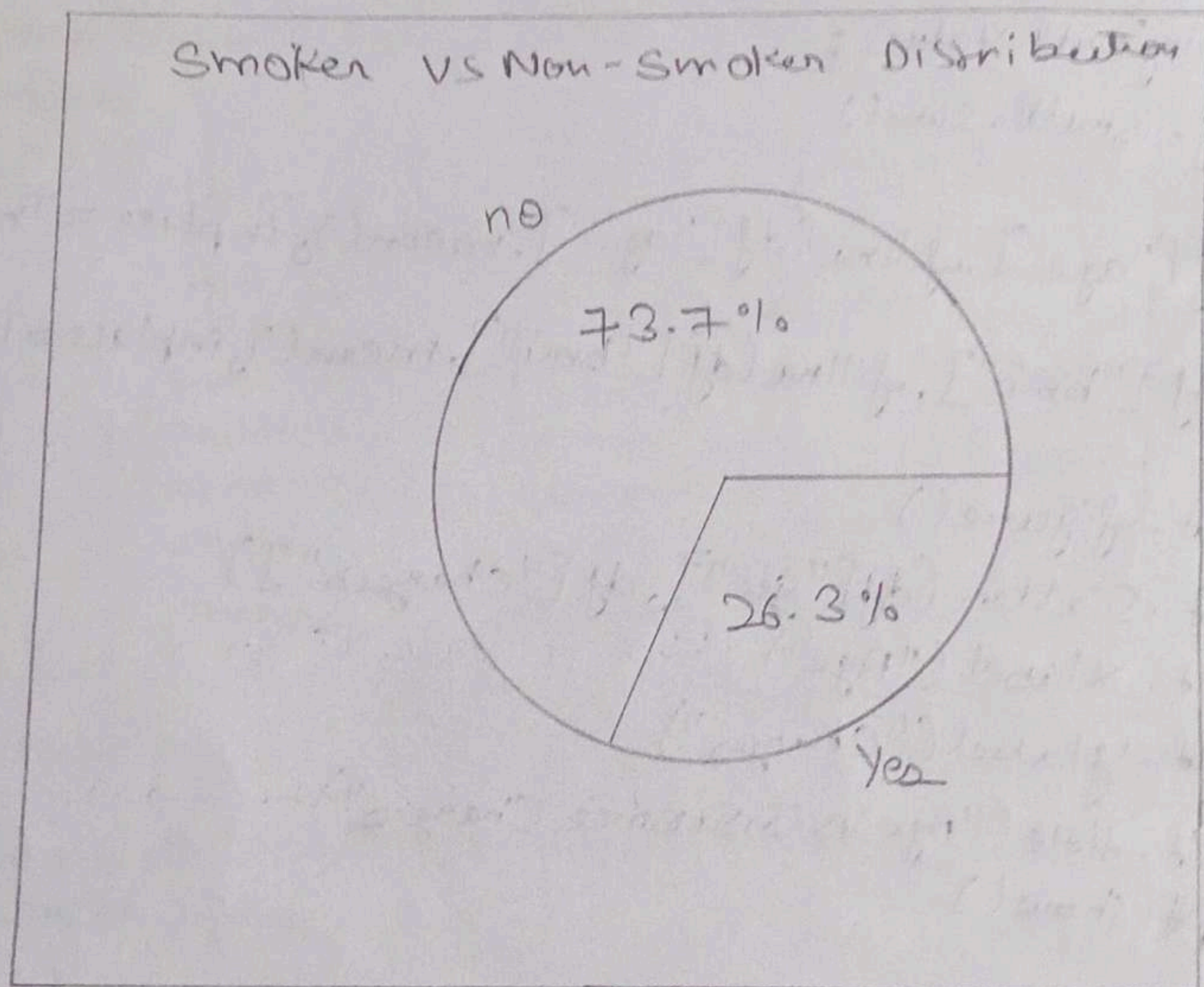



```
plt.figure()
```

```
plt.pie(smoker_counts.values, labels=smoker_counts.index,  
autopct="%1.1f%%")
```

```
plt.title("Smoker vs Non-Smoker Distribution")
```

```
plt.show()
```



```
plt.figure()
```

```
plt.bar(avg_smoker.index, avg_smoker.values)
```

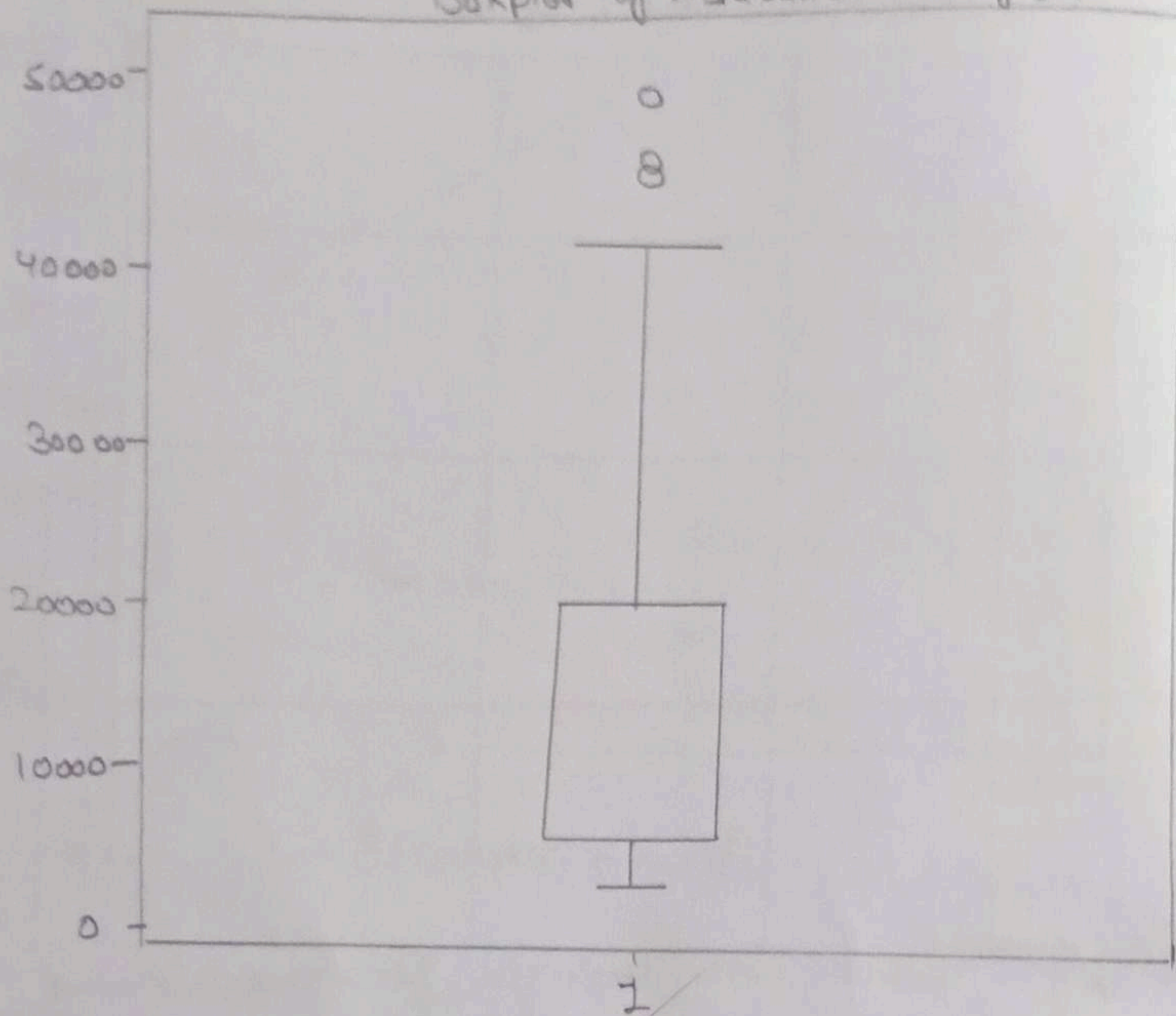
```
plt.xlabel("Smoker")
```

```
plt.ylabel("Average charges")
```

```
plt.title("Average charges: Smoker vs non-smoker")
```

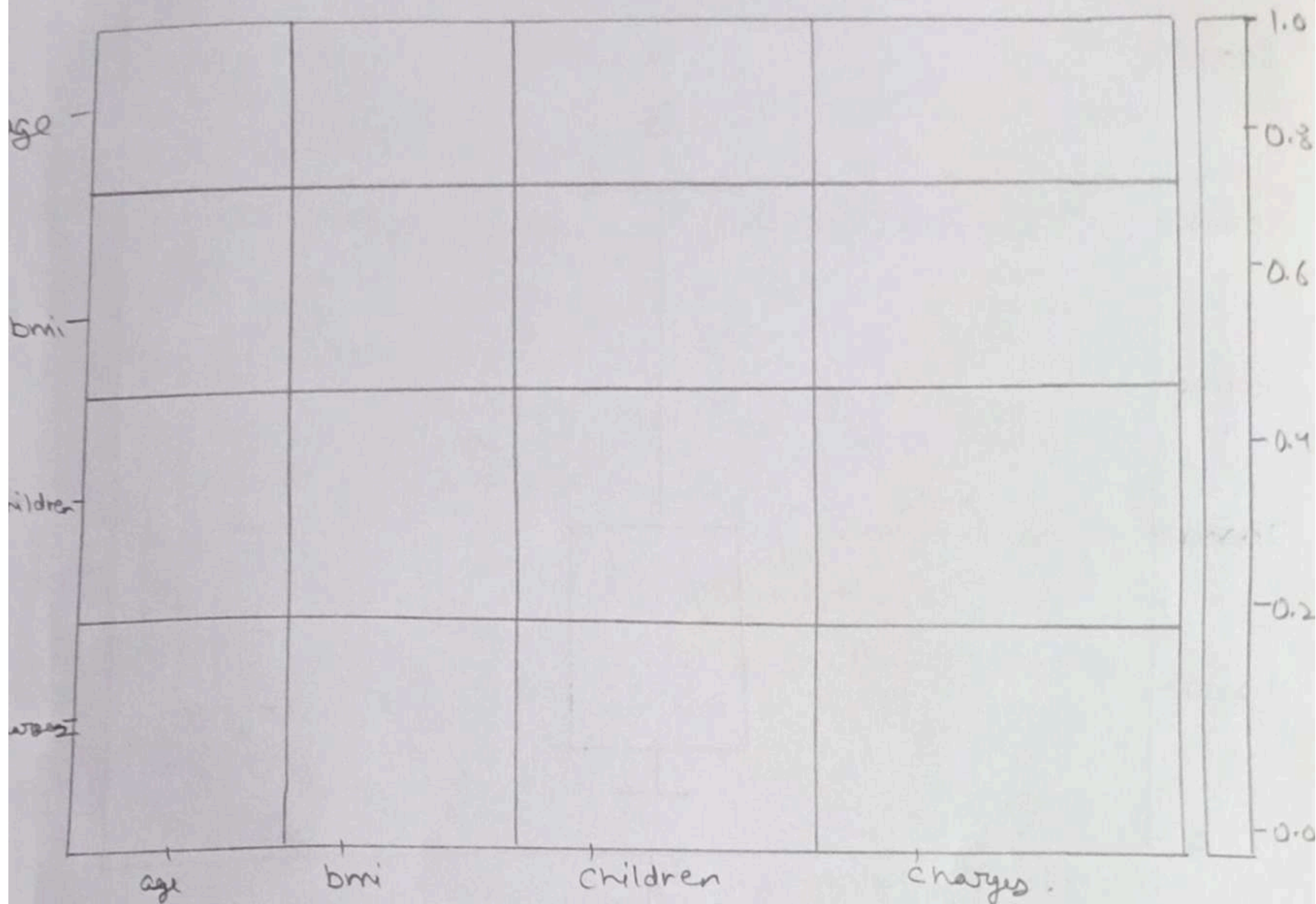
```
plt.show()
```


Boxplot of Insurance charges.



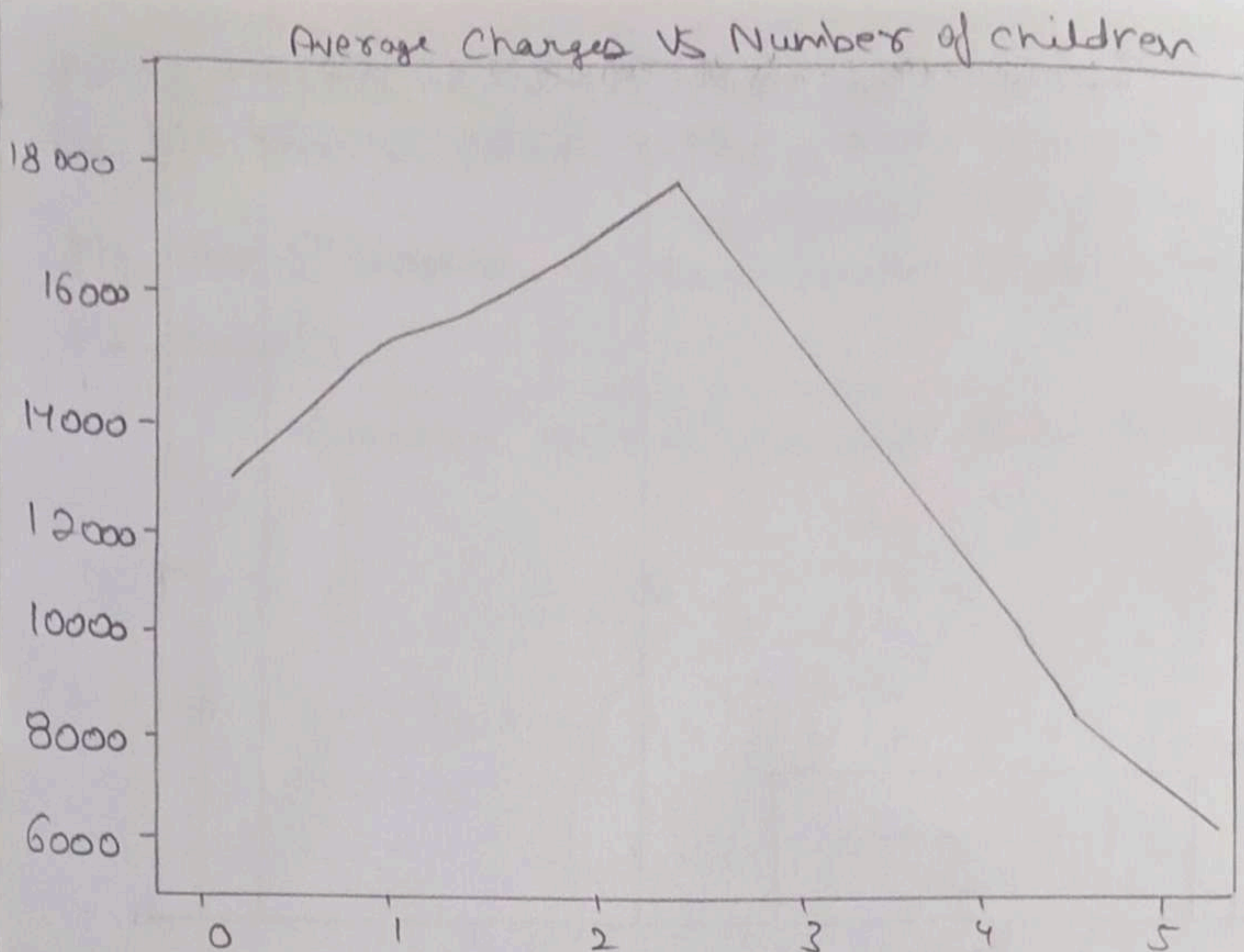
Boxplot of Insurance charges. The boxplot shows the distribution of insurance charges. The median is approximately 14,000. The first quartile (Q1) is approximately 5,000 and the third quartile (Q3) is approximately 20,000. The whiskers extend from approximately 2,000 to 41,000. There are two outliers at approximately 48,000 and 50,000.

Correlation Heatmap of Insurance Datasets



```
plt.figure()  
plt.boxplot(df["charges"])  
plt.ylabel("charges")  
plt.title("Boxplot of Insurance charges")  
plt.show()
```



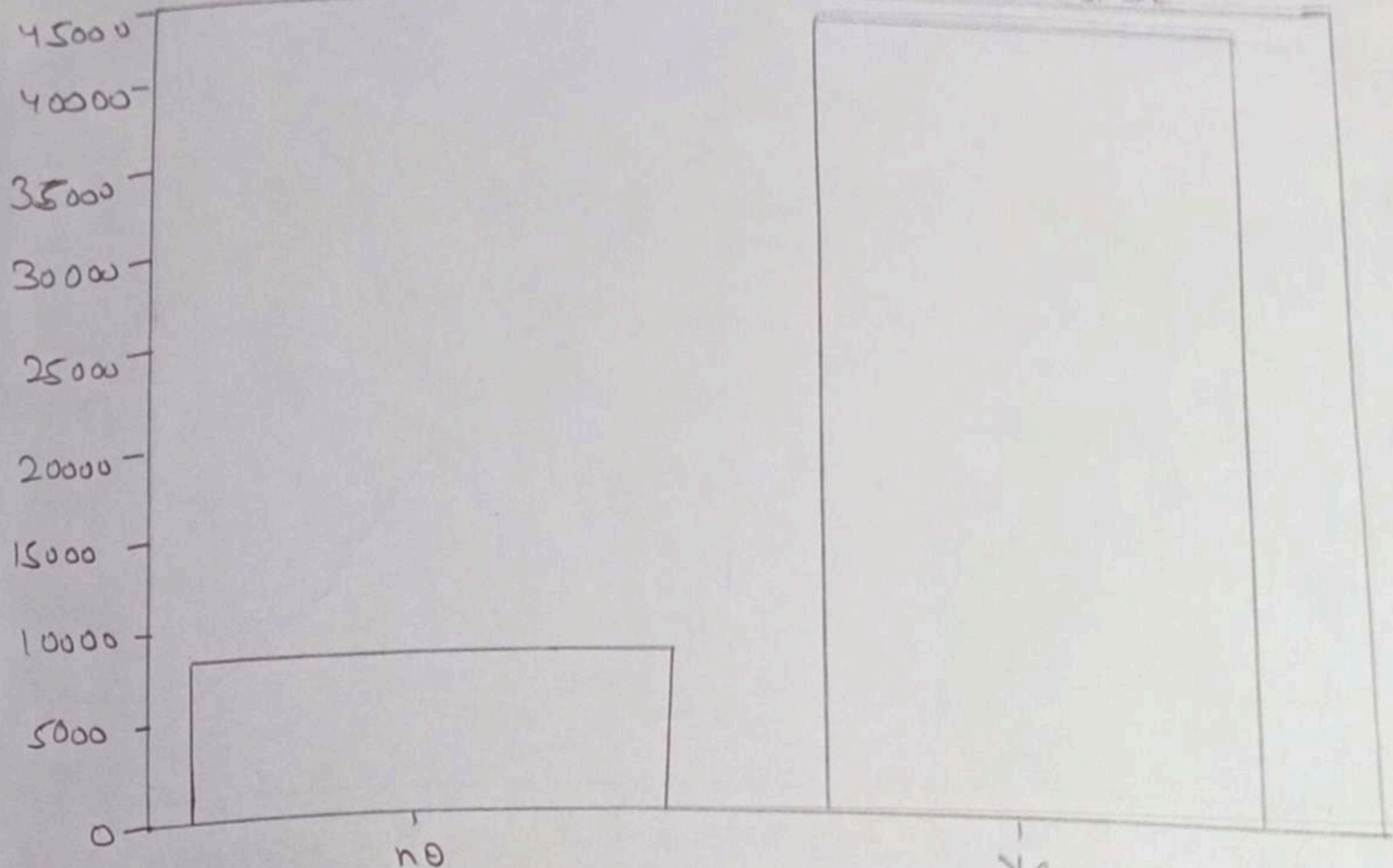


```

numeric_df = df.select_dtypes(include=[np.number])
corr = numeric_df.corr()
plt.figure()
plt.imshow(corr)
plt.colorbar()
plt.xticks(range(len(corr.columns)), corr.columns, rotate=45)
plt.yticks(range(len(corr.columns)), corr.columns)
plt.title("Correlation Heatmap of Insurance Data sets")
plt.show()

```


Average Charges: Smoker VS Non-Smoker



```

avg-children = df.groupby("children")["charges"].mean()
plt.figure()
plt.plot(avg-children.index, avg-children.values)
plt.xlabel("Number of children")
plt.ylabel("Average charges")
plt.title("Average charges vs Number of children")
plt.show()
    
```


Q Create a dataset related to Medical Sales having columns such as customerid, customer name, name of product, cost of product, quantity of product, mode of payment.

Ans) import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
df = pd.read_csv("Sales.csv")
df.

	Cust-ID	Cust-Name	P-Name	P-cost	P-Quantity
0	01	Ayush	Trimmer	700.0	2.0
1	02	Ritik	Cookies	450.0	5.1
2	03	NAN	Cookies	450.0	4.0
3	04	BOB	Cereals	500.0	8.0
4	05	Joe	Books	499.0	4.0
5	06	Kiran	Toothpaste	799.0	NAN

Pay-mode

Cash

UPI

Cash

Credit Card

Cash

Credit Card

df1 = df.copy()

df1.info()

<class 'Pandas.core.frame.DataFrame'>

RangeIndex: 7 entries, 0 to 6

Data columns (Total 6 columns):

#	Column	Non-Null Count	Dtype
0	Cust-ID	6 non-null	int64
1	Cust-Name	5 non-null	Object
2	P-Name	6 non-null	Object
3	P-cost	6 non-null	float64
4	P-Quantity	5 non-null	float64
5	Pay-mode	6 non-null	Object

dtypes: float64(2), int64(1), Object(3)

memory usage: 468.0+ bytes.

df.describe()

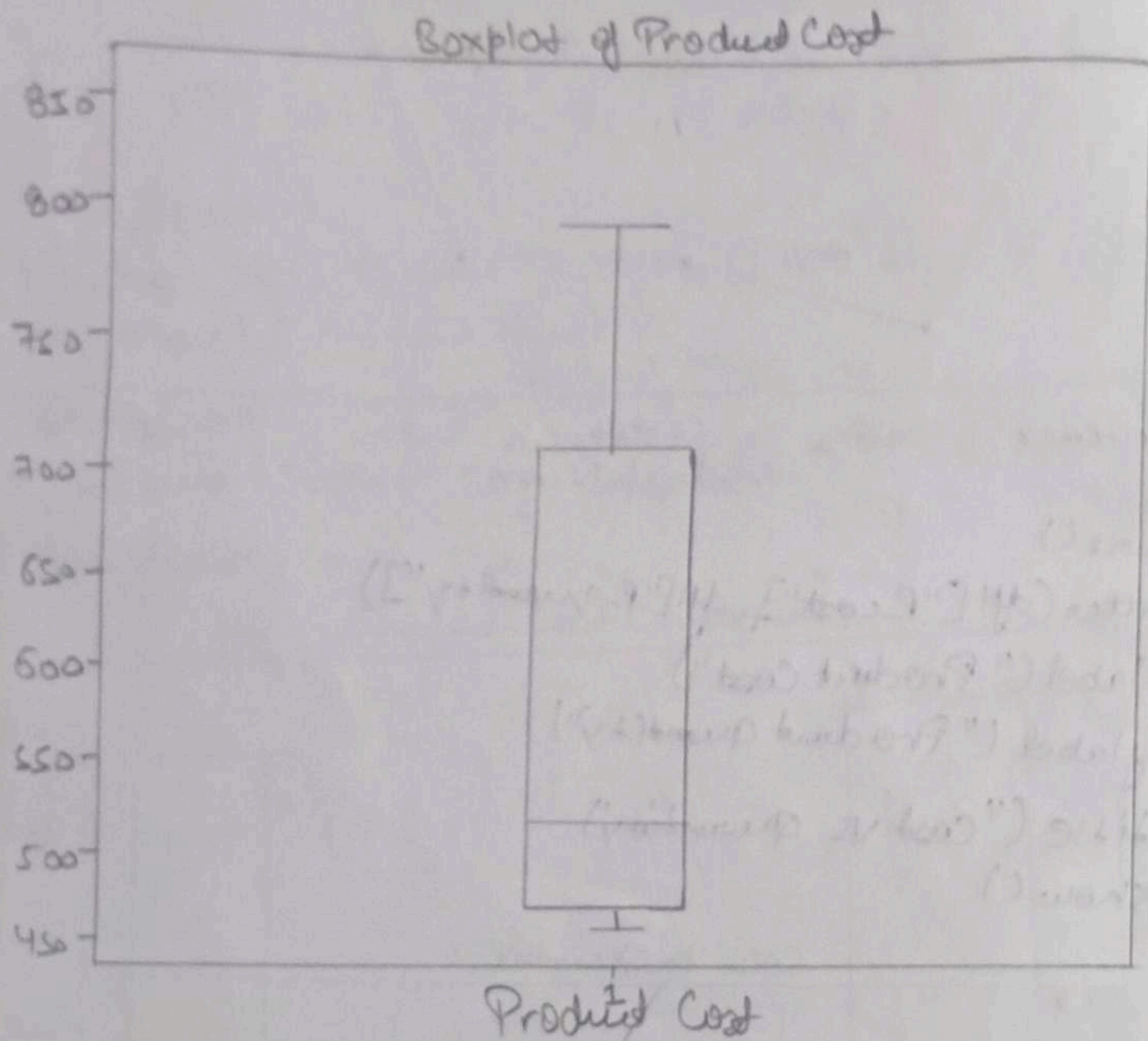
Output:

	Cust-ID	P-cost	P-Quantity
Count	7.000000	6.000000	6.000000
Std	2.160247	44.46909	1.632993
mean	104.000000	82.500000	3.333333
min	101.000000	43.000000	1.000000
25%	102.500000	52.500000	2.250000
50%	104.000000	65.000000	3.500000
75%	105.500000	100.000000	4.750000
max%	107.000000	80.000000	5.000000


```

plt.figure()
plt.boxplot(df["P_cost"])
plt.xlabel("Product Cost")
plt.title("Boxplot of Product Cost")
plt.show()

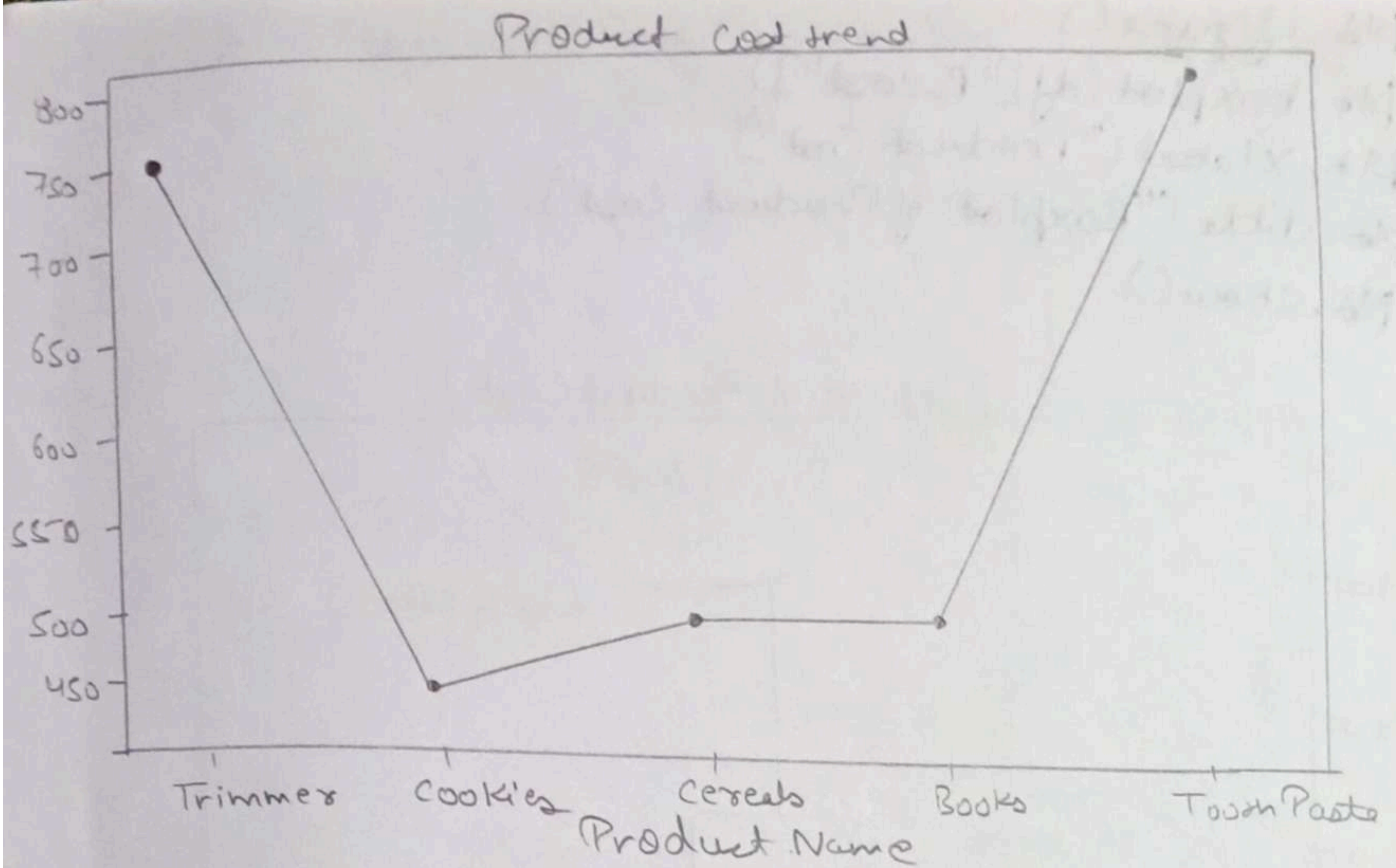
```



```

Payment_counts = df["Pay_mode"].value_counts()
plt.figure()
plt.pie(payment_counts, labels = payment_counts,
        index, autopct = "%0.1f%%")
plt.title("Payment Mode Distribution")
plt.show()

```

plt.figure()

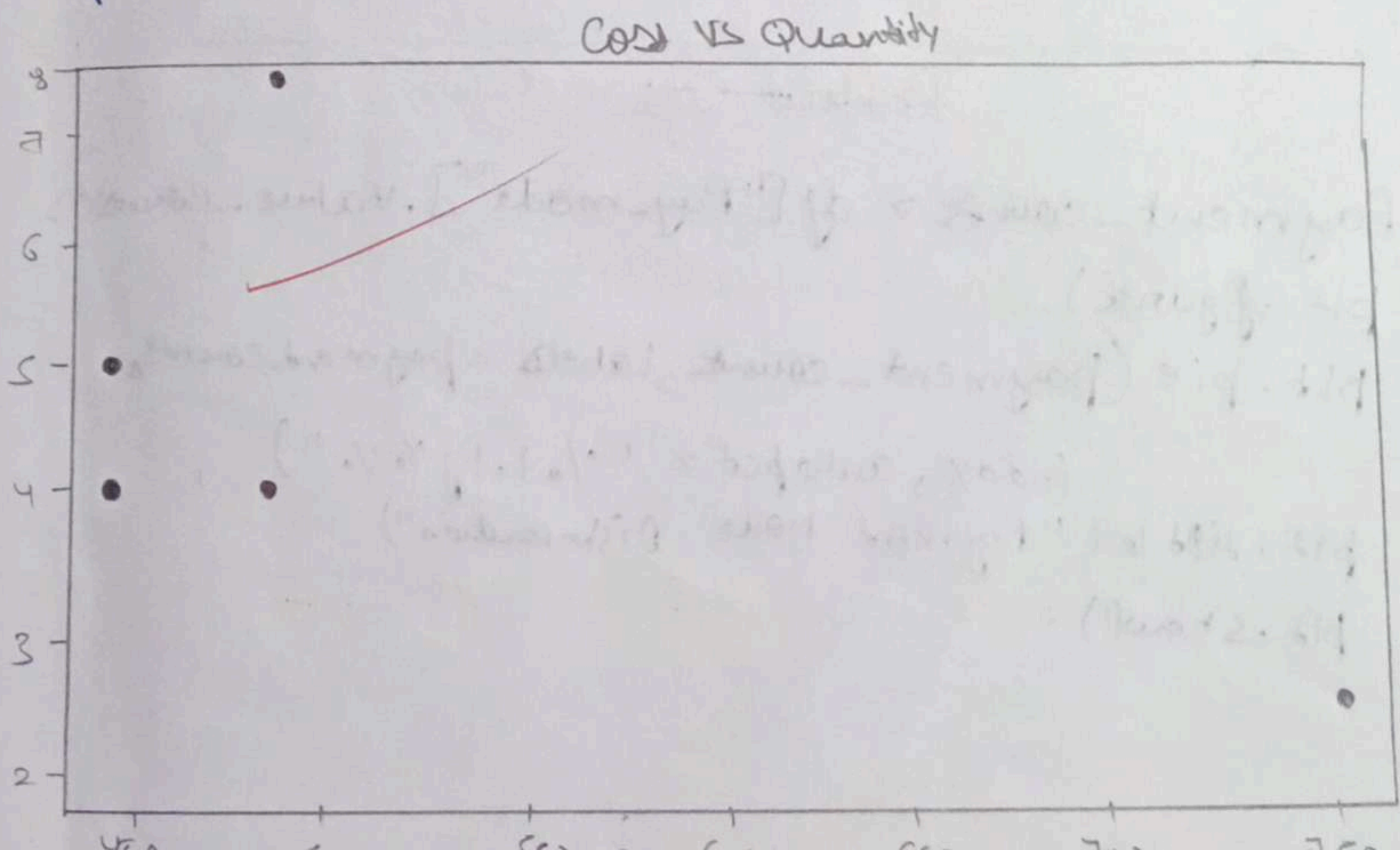
plt.scatter(df["P-cost"], df["P-quantity"])

plt.xlabel("Product Cost")

plt.ylabel("Product Quantity")

plt.title("Cost vs Quantity")

plt.show()



	Cust_ID	Cust_Name	P_Name	P_Cost	P_Quantity	Pay_men
0	101	Ayush	Trimmer	750	2.0	Cost
1	102	Piyush	Cookies	450	5.0	Up
2	103	X	Cookies	450	4.0	Cost
3	104	Bob	Cereals	500	8.0	Credit
4	105	Joe	Books	499	4.0	Cost
5	106	Kiran	Tooth Paste	799	4.6	Credit
6	107				10.0	

```
plt.figure()
```

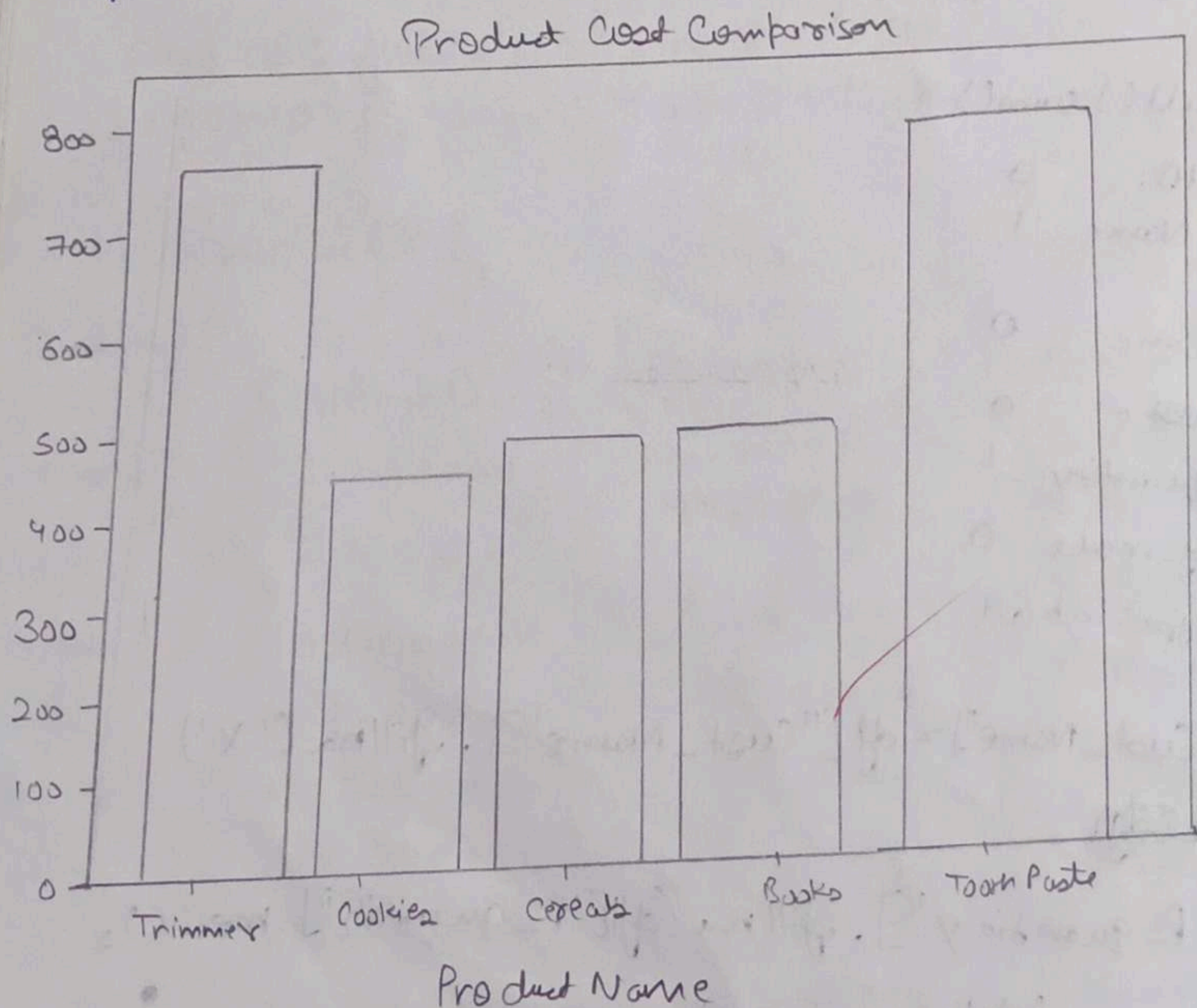
```
plt.bar(df1["P_Name"], df1["P_Cost"])
```

```
plt.xlabel("Product Name")
```

```
plt.ylabel("Product Cost")
```

```
plt.title("Product Cost Comparison")
```

```
plt.show()
```



df.head()

	cust-ID	cust-Name	P-Name	P-cost	P-quantity	Pay-mode
0	1	Ayush	Trimmer	750	2.0	Cash
1	2	Ritik	Cookies	450	5.0	UPI
2	3	NAN	Cookies	450	4.0	Cash
3	4	Bob	Cereals	500	8.0	Credit Card
4	5	Joe	Books	499	4.0	Cash

df.tail()

	cust-ID	cust-Name	P-Name	P-cost	P-quantity	Pay-mode
1	2	Ritik	Cookies	450	5	UPI
2	3	NAN	Cookies	450	4	Cash
3	4	Bob	Cereals	500	8	Credit Card
4	5	Joe	Books	499	4	Cash
5	6	Kiran	Tooth Paste	799	NAN	Credit Card

df.isnull().sum()

```

cust-ID      0
cust-Name    1
cost         0
P-Name       0
P-cost       0
P-quantity   1
Pay-mode     0
dtype: int64

```

df["cust-Name"] = df["cust-Name"].fillna('X')

~~print(df)~~

df["P-quantity"] = df["P-quantity"].mean(),

inplace = True)

df

7 engine-capacity 15 non-null float64
 8 price (Lakhs) 15 non-null float64

dtypes: float64(4), int64(1), object(4)

memory usage: 1.5+ KB

	S.No	mfg-year	milage	engine-capacity	price (Lakhs)
Count	20.000	15.000	15.00	15.00	15.00
mean	10.500	2020.93	15.20	267.60	52.80
Std	1.00	4.99	5.29	103.00	30.06
min	5.75	2010.00	8.00	150.00	3.00
25%	10.5-75	2019.50	12.00	200.00	35.00
50%	10.50	2022.00	14.00	225.00	50.00
75%	15.25	2024.50	17.50	307.50	65.00
max	20.00	2026.00	30.00	480.00	125.00

df.head()

	S.No	Car_mfg	Car_model	Type	mfg-year	colour	milage
0	1	toyota	inova	diesel	2023	red	20
1	2	maruti-suzuki	NAN	cng	2025	250	25
2	3	bmw	petrol	petrol	2024	NAN	NAN
3	4	audi	audi	NAN	NAN	400	65
4	5	Jeep	diesel	diesel	black	150	50

df.tail()

df.sample(5)

df.shape

(20, 8)

milage	engine_capacity	price (LAKHS)
20	200	20
30	250	25
NaN	NaN	NaN
10	400	65
8	150	50
NaN	200	65
12	225	75
13	NaN	40
14	330	30
11	265	NaN
NaN	260	3
12	NaN	40
18	480	60
19	NaN	NaN
17	450	125
15	NaN	NaN
NaN	200	90
14.0	199	NaN
15	185	36
	200	40
		15
		16

df.info()

df.describe()

<class 'Pandas.core.frame.DataFrame'>

RangeIndex: 20 entries, 0 to 19

Data columns (total 9 columns):

#	Column	Non-Null Count	Dtype
0	S.NO	20 non-null	int64
1	Car_mfg	20 non-null	Object
2	Car_model	15 non-null	Object
3	Type	16 non-null	Object
4	mfg-year	15 non-null	Object
5	colour	15 non-null	float64
6	milage	15 non-null	Object

Q Create a datasets named car Sales which contain information related to a car manufacture
 Car-model, Car-type, Car-price, manufacture date, milage, no. of seats, ~~App~~ Engine Capacity (cc)
 @name of Car manufacture

import pandas as pd

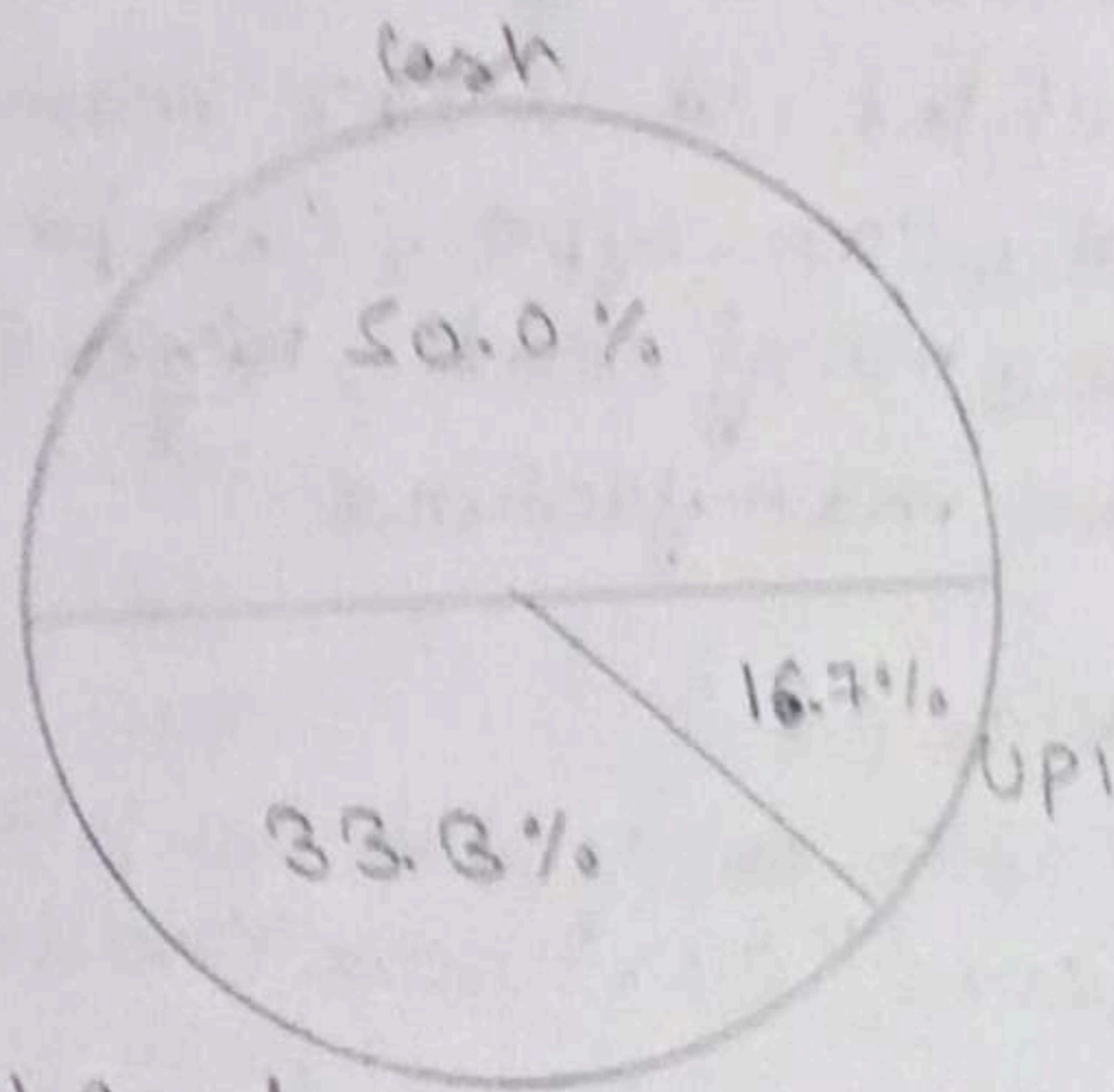
import matplotlib.pyplot as plt

df = pd.read_csv("C:\\Users\\Vidit\\One Drive\\Desktop\\car.csv")

df

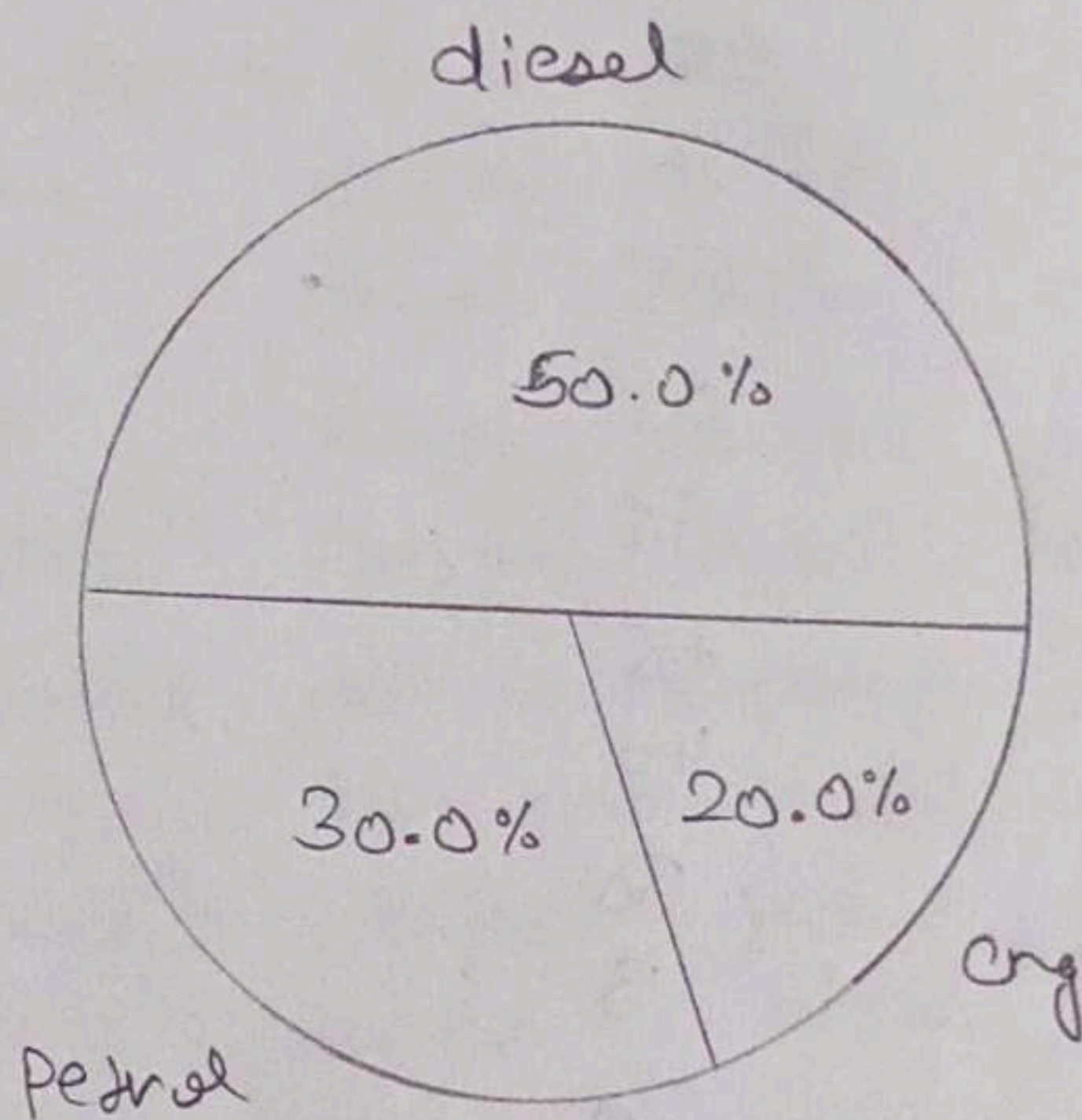
SNO	Car_mfg	Car_model	Type	mfg-year	Color
0 1	Toyota	innova	diesel	2023	White
1 2	maruti-suzuki	NAN	Cng	2025	NAN
2 3	bmw	Creta	NAN	2024	black
3 4	audi	audi	diesel	NAN	NAN
4 5	jeep	NAN	NAN	2024	black
5 6	ferrari	diesel ferrari	petrol	2021	Pink
6 7	mini-cooper	mini-cooper	petrol	NAN	Yellow
7 8	range-rover	range-rover	diesel	2022	blue
8 9	Safari	Safari	Cng	2025	Cream
9 10	maruti-suzuki	maruti-suzuki	petrol	NAN	NAN
10 11	RR	RR	petrol	2022	brown
11 12	NAN	crystal	diesel	NAN	white
12 13	POLO	polo	NAN	NAN	black
13 14	NAN	Toyota	petrol	2019	NAN
14 15	Verna	Verna	Cng	2020	NAN
				2026	Silver

Payment Mode Distribution



Credit card

Distribution of cars by fuel type.



```
df_sorted = df2.sort_values("mfg-year")
plt.figure()
plt.plot(df_sorted["mfg-year"], df_sorted
         ["price(LAKHS)"], marker='o')
plt.xlabel("Manufacturing Year")
plt.ylabel("Price (LAKHS)")
plt.title("Price Trend over Years")
plt.show()
```


engine-capacity price(LAKHS)

200	20
250	25
267.6	20
400	65
150	50
200	65
225	75
225	40
267.6	30
380	3
265	20
265	40
260	60
276	20
200	125
199	20
185	30
200	3
480	90
225	20
350	56
400	48

```
plt.pie(df2["Type"].value_counts(), labels=df2["Type"],
        value_counts().index, autopct="%1.1f%%")
plt.title("Distribution of cars by fuel type")
plt.show()
```


0	1	Toyota	innova	diesel	2023	white
1	2	maruti_suzuki	base	crng	2026	white
2	3	bmw	creta	petrol	2024	black
3	4	audi	audi	diesel	2026	pink
4	5	jeep	base	diesel	2024	blue
5	6	ferari	ferari	diesel	2021	Yellow
6	7	mini-cooper	mini-cooper	petrol	2026	Cream
7	8	range-rover	range-rover	petrol	2022	white
8	9	safari	safari	petrol	2028	brown
9	10	RR	rr RR	diesel	2026	white
10	11	Crysta	base	crng	2022	black
11	12	polo	po base	petrol	2026	white
12	13	toyota	base verna	diesel	2026	green
13	14	Verma	lexis	petrol	2023	cream
14	15	lexis	base	crng	2019	black
15	16	horr	base	petrol	2012	blue
16	17	Skoda	sk Skoda	crng	2015	silver
17	18	hata	hata	diesel	2010	silver
18	19	maruti_suzuki	maruti_suzuki	diesel	2012	black
19	20	hata	hata	base crng	2013	green
					2020	silver


```
df2 = df.copy()
df2.isnull().sum()
```

S.No	0
Car_mfg	0
Car_model	5
Type	4
mfg-year	5
colour	5
milage	5
engine-capacity	5
price (LAKHS)	5

```
dtype: int64
```

```
age_milage = df2["milage"].mean()
```

```
df2["milage"] = df2["milage"].fillna(age_milage)
```

```
df2["colour"] = df2["colour"].fillna("white")
```

```
df2["Car_model"] = df2["Car_model"].fillna("base")
```

```
type = df2["Type"].mode()[0]
```

```
df2["Type"] = df2["mfg-year"].fillna("2026").astype("int64")
```

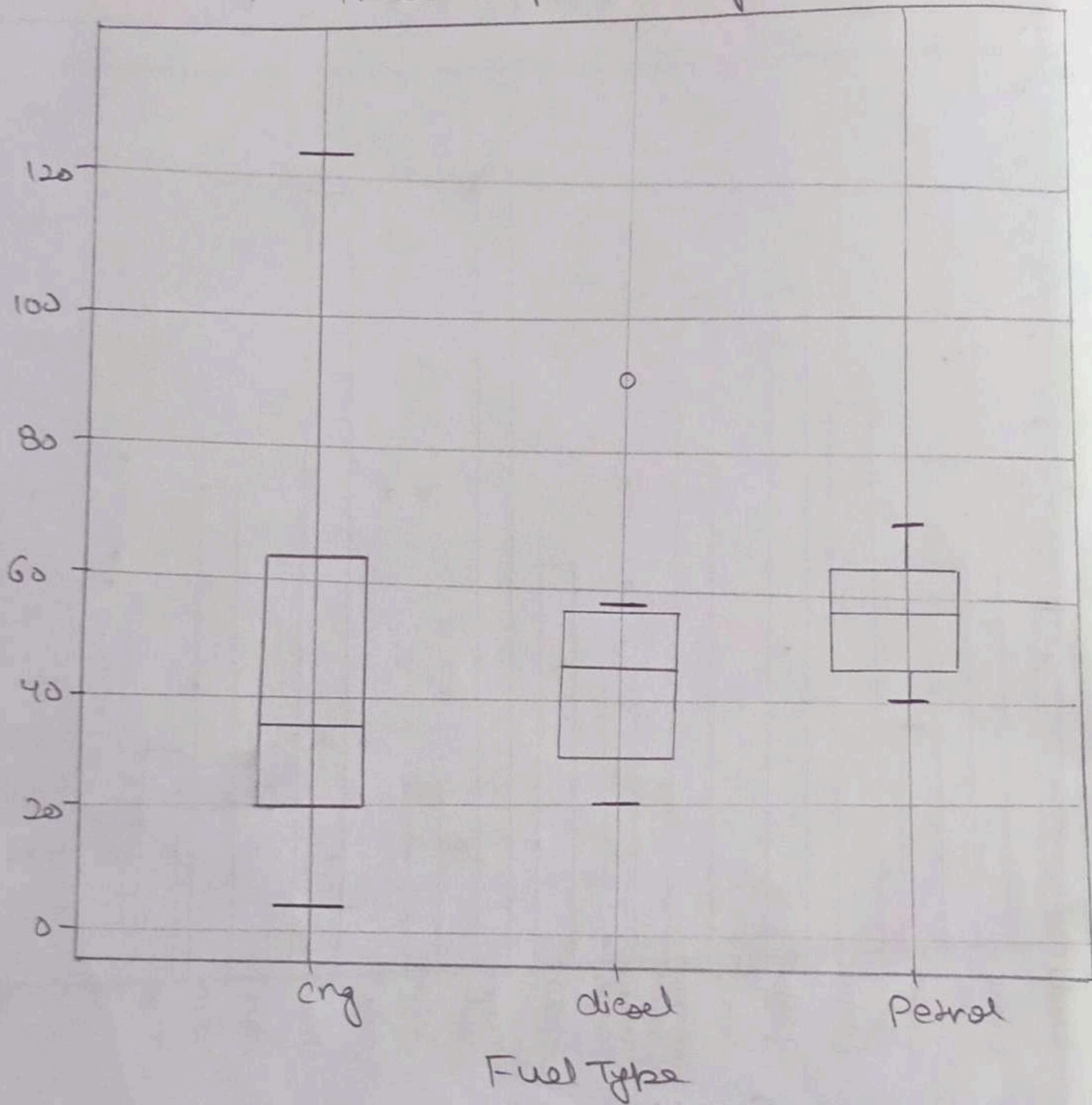
```
engine_capacity = df2["mfg-year"].fillna("2026").astype("int64")
```

```
df2["Engine_capacity"] = df2["Engine_capacity"].fillna(0).astype("int64")
```

```
df2["engineprice(LAKHS)"] = df2["price(LAKHS)"].fillna(0)
```

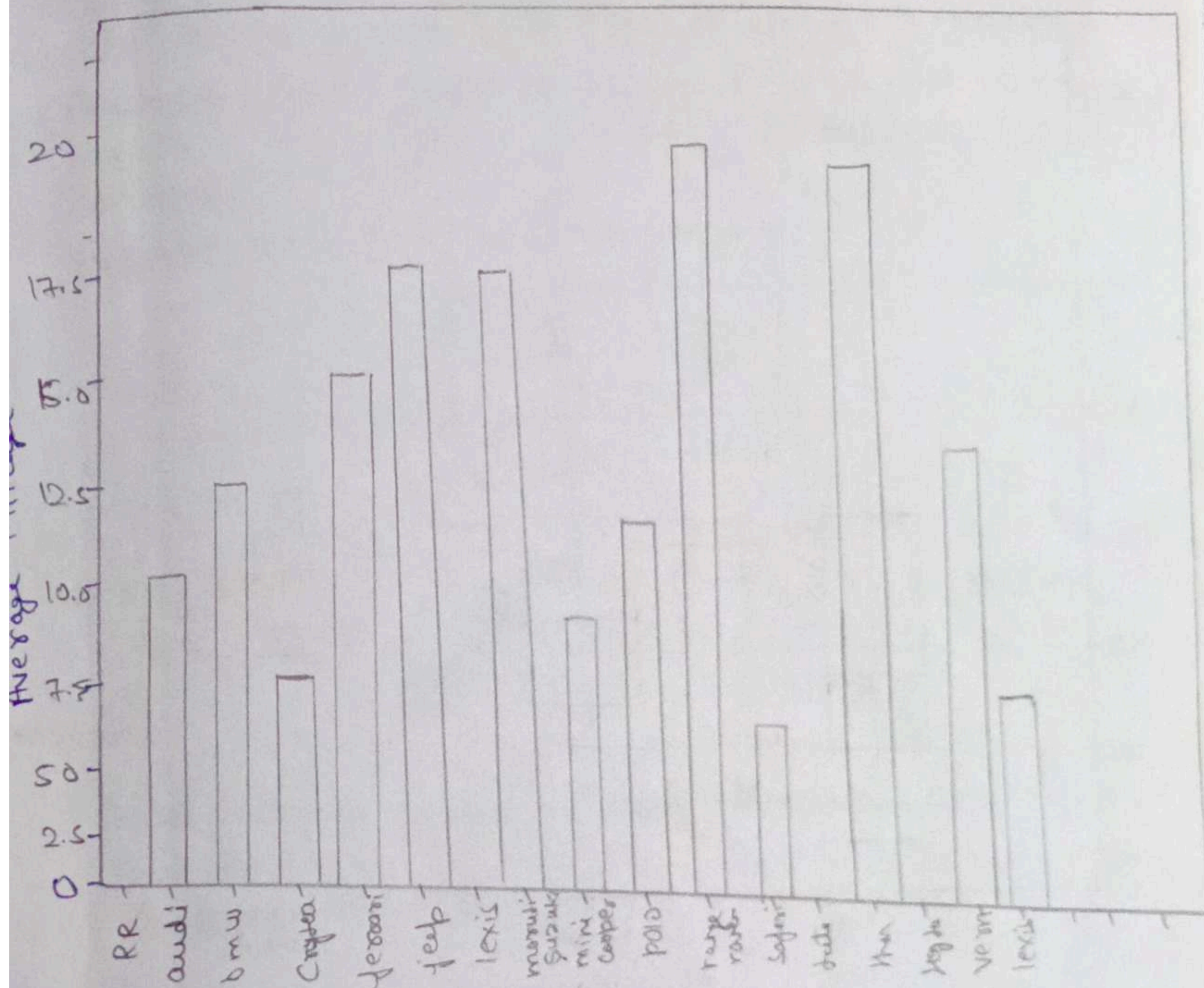
```
df2.
```


Price Comparison by Fuel Type



[Signature]
06/02/24

Average Mileage by Manufacturing

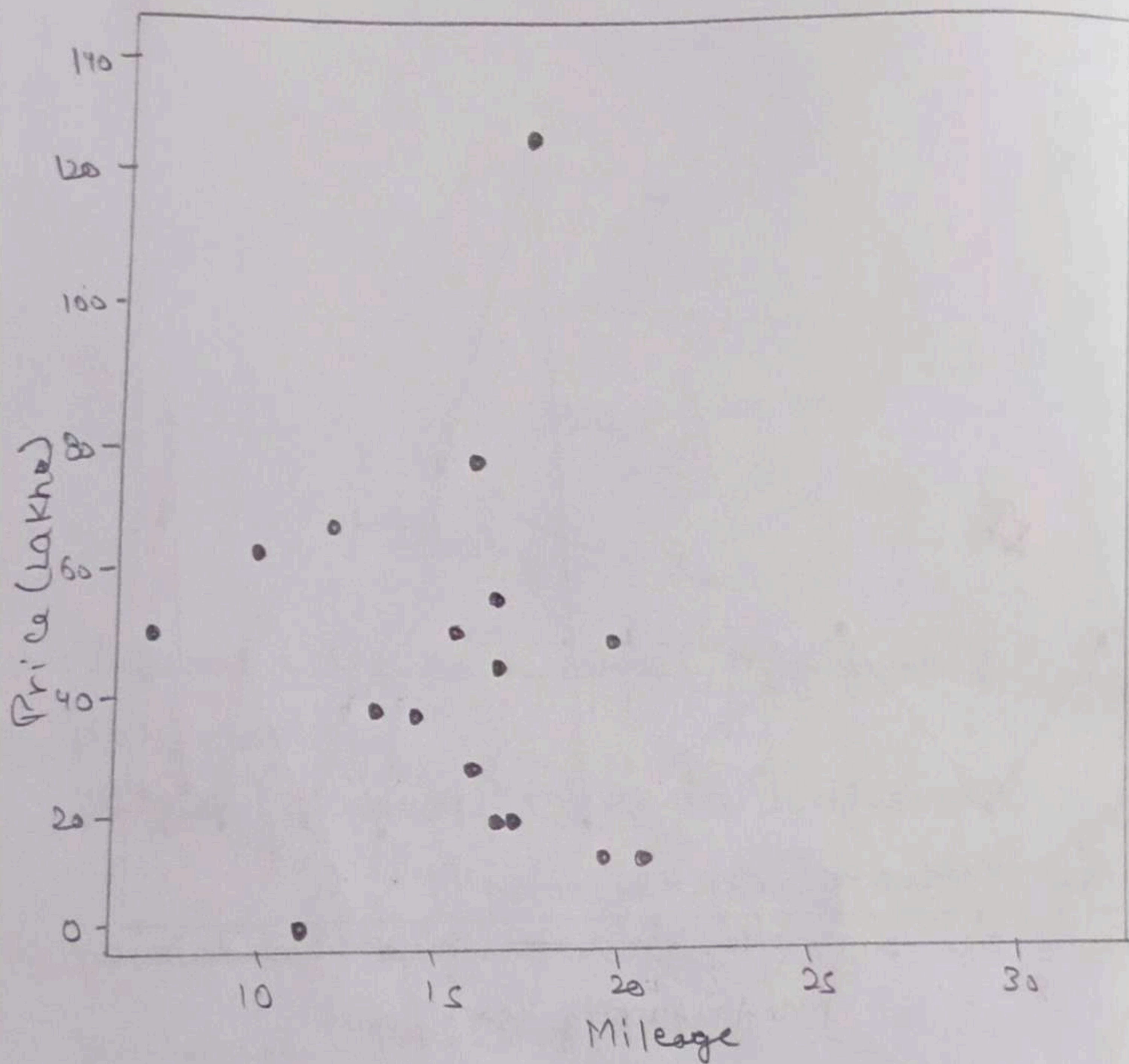


Car Manufacturing

```

df2.boxplot(column="Price (LAKHS)", by="Type")
plt.title("Price Comparison by Fuel Type")
plt.subtitle("")
plt.xlabel("Fuel Type")
plt.ylabel("Price (Lakhs)")
plt.show()
    
```

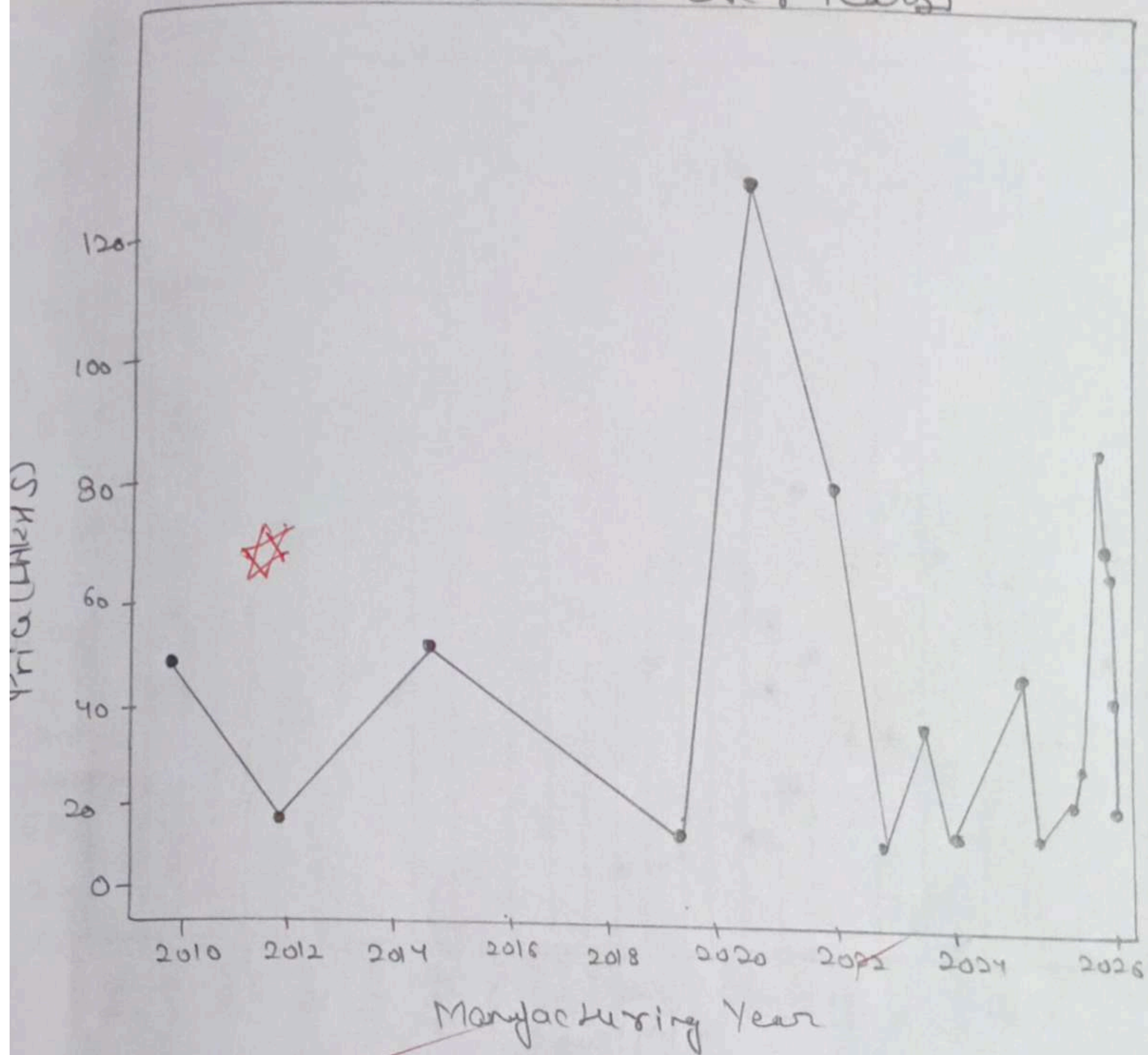

Mileage vs Price



```

avg_milage = df.groupby("Car_mfg")["milage"].mean()
plt.figure(figsize=(6,4))
plt.bar(avg_milage.index, avg_milage.values)
plt.xlabel("Car Manufacturing")
plt.ylabel("Average Mileage")
plt.title("Average Mileage by Manufacturer")
plt.xticks(rotation=45)
plt.show()
    
```


Price Trend Over Years



plt.figure()

plt.scatter(df2["mileage"], df2["price(LAKHS)"])

plt.xlabel("Mileage")

plt.ylabel("Price (LAKHS)")

plt.title("Mileage vs Price")

plt.show()

Ans
n = int(input("Enter number of nodes:"))

graph = {}

for i in range(n):

node = input("Enter node:").

neighbours = input("Enter neighbours separated
by space:").split()

graph[node] = neighbours

Root_node = input("Enter the Root Node:")

Goal_node = input("Enter the Goal Node:")

print(graph)

~~def~~ def bfs(graph, ~~nodes~~ graph, start, goal):

visited = []

queue = []

visited.append(start)

queue.append(start)

while queue:

m = queue.pop(0)

print(m, end = " ")

if m == Goal_node:

print(f"\n Goal node found - {Goal_node}")

return

for neighbour in graph[m]:

if neighbour not in visited:

visited.append(neighbour)

queue.append(neighbour)

Print("\n Goal node not found")


```
graph = {  
    "A": ["C", "B"],  
    "B": ["D", "E", "F"],  
    "C": ["F", "D"],  
    "D": ["E", "F"],  
    "E": ["F"],  
    "F": []  
}
```



```
def bfs(graph, Node):
```

```
    visited = []
```

```
    queue = []
```

```
    visited.append(Node)
```

```
    queue.append(Node)
```

```
    while queue:
```

```
        m = queue.pop(0)
```

```
        print(m)
```

```
        for n in graph[m]:
```

```
            if n not in visited:
```

```
                visited.append(n)
```

```
                queue.append(n)
```

```
bfs(graph, "A")
```



```

n = int(input("Enter number of nodes :-"))
graph = {}
for i in range(n):
    node = input("Enter node:")
    neighbours = input("Enter neighbours separated  
by space: ").split()
    graph[node] = neighbours

```

```

root_node = input("Enter The Root Node:")
goal_node = input("Enter the Goal Node:")
visited = set()

```

```

def dfs(graph, node, goal_node):
    if node in visited:
        return False

    print(node, end=" ")
    visited.add(node)

    if node == goal_node:
        print("\n Goal node found - {goal_node}")

    for neighbour in graph[node]:
        if dfs(graph, neighbour, goal_node):
            return True

    return False

```

```

Print("DFS Traversal Order:-")
dfs(graph, root_node, goal_node)

```


Ans)

```
graph = {  
    "A": ["C", "B"],  
    "B": ["D", "E", "F"],  
    "C": ["F", "D"],  
    "D": ["E", "F"],  
    "E": ["F"],  
    "F": []  
}
```

visited = set()



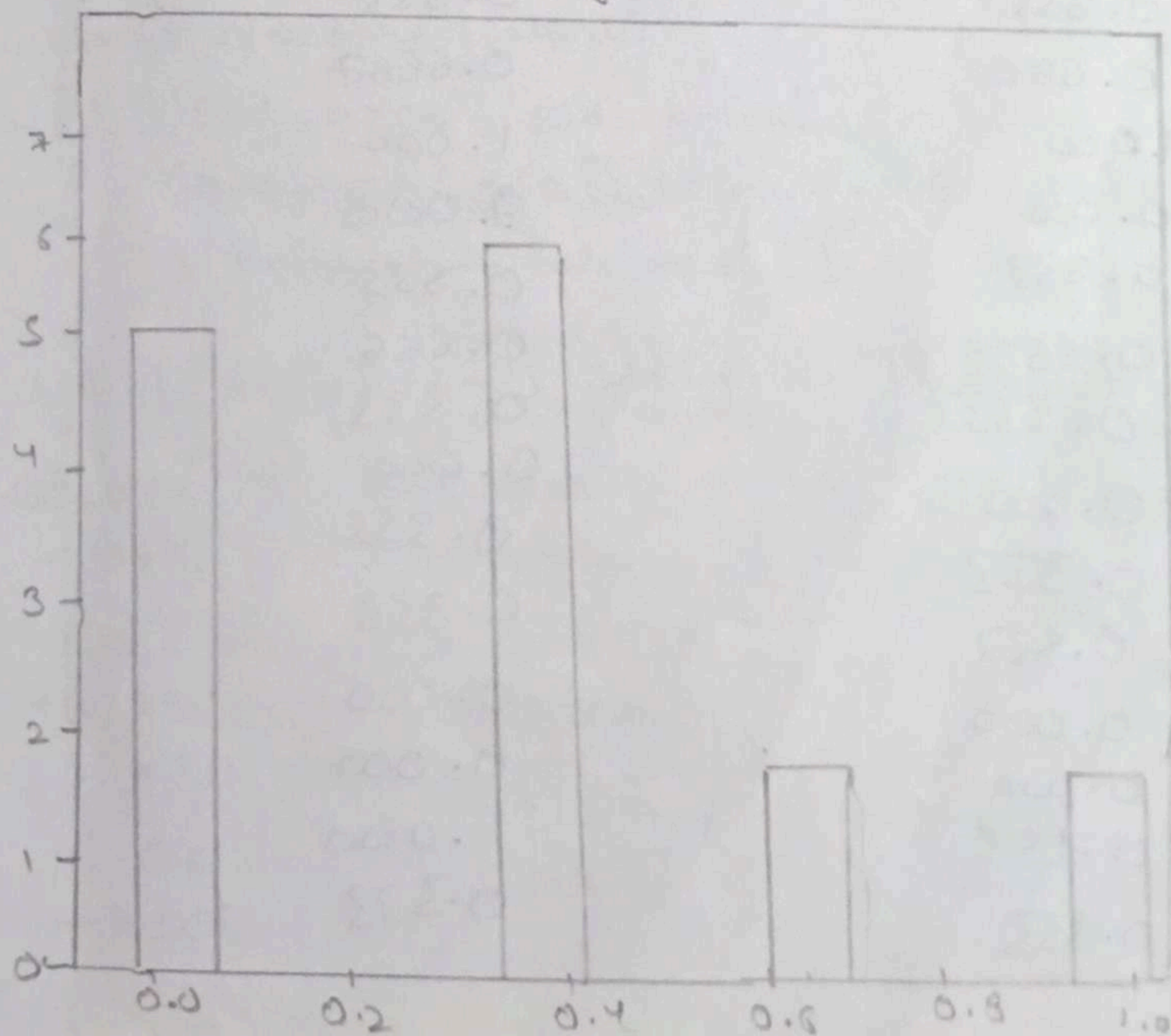
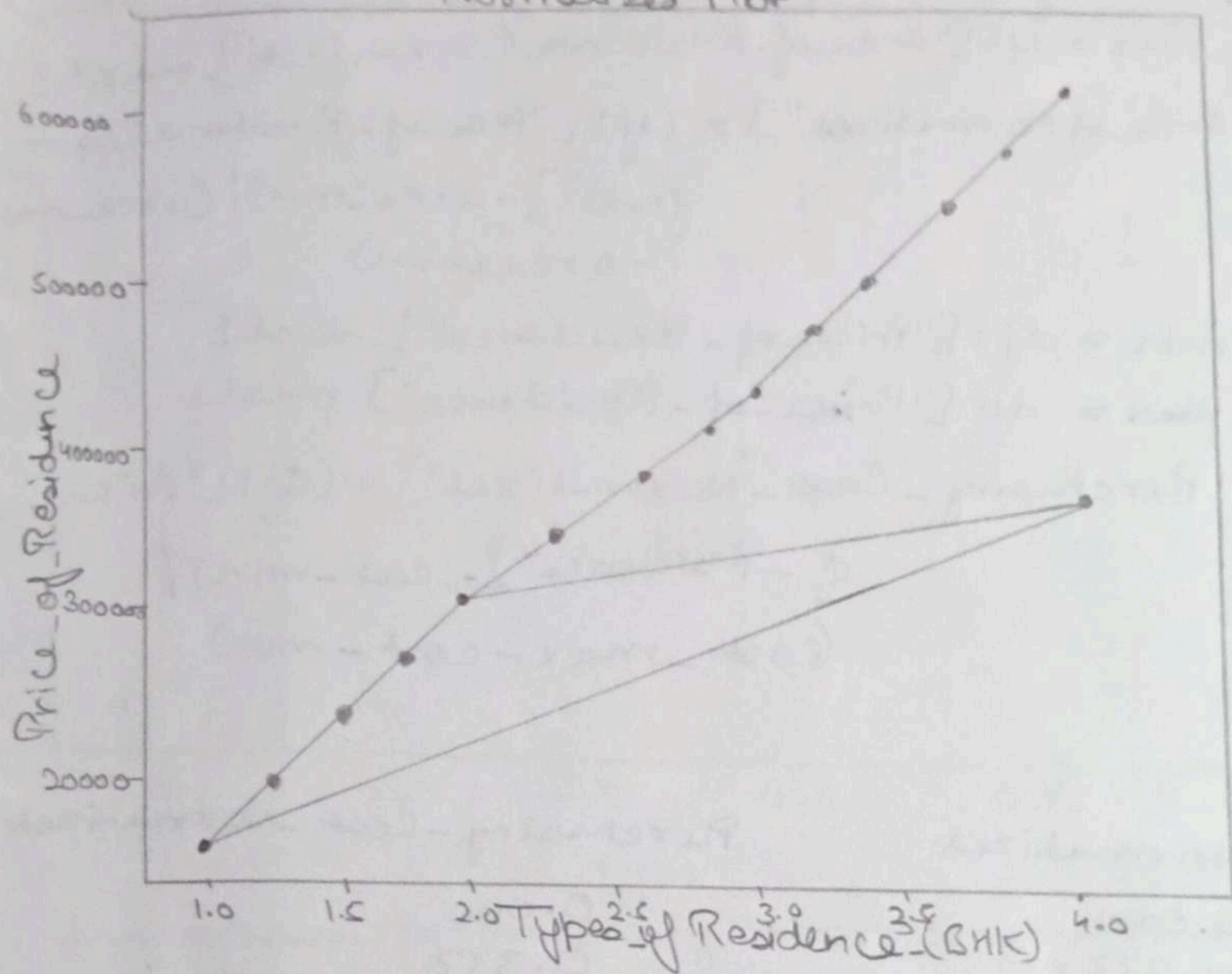
```
def dfs(graph, Node, visited):  
    if Node not in visited:  
        visited.add(Node)  
        print(visited)  
        for n in graph[Node]:  
            dfs(graph, n, visited)
```

dfs(graph, "A", visited)

Output

{'A', 'B', 'D', 'C', 'F', 'E'}

Normalized Plot



"Price-of-Residence": [150000, 300000, 450000, 600000,
150000, 300000, 450000, 300000, 150000, 300000,
300000, 150000, 400000, 300000],

"No.-of-Floor": [2, 3, 8, 2, 6, 4, 2, 1, 2, 3, 2]
}

df = pd.DataFrame(info)

df

df1 = df.copy()

Scale = pd.DataFrame(MinMaxScaler().fit_transform(
df1[['Price-of-Residence', 'Types-of-Residence-
[]])

Print(Scale)

plt.figure()

plt.plot(df1['Type-of-Residence-(BHK)'], df1['Price-of-
Residence'], marker='o')

plt.xlabel('Type-of-Residence-(BHK)')

plt.ylabel('Price-of-Residence')

plt.title('Normalized plot')

plt.show()

Q) Create a data set related to medical/health care which has column such as person ID, Name, Diagnosis, Sugar level, BP, Average income, Age.
Normalize cost and area of property.
Apply $\rightarrow$ Min-Max normalization.

Q) Create a data set related to medical/health care which has column such as person ID, Name, Diagnosis, Sugar level, BP, Average income, Age.

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
from sklearn.preprocessing import MinMaxScaler  
Scaler = MinMaxScaler()
```

```
# DataFrame
```

```
info = {
```

```
    "Types of Residence (BHK)": [1, 2, 3, 4, 5, 6, 7, 2, 1, 2, 3, 4],
```

```
    "Area of Residence (sq-ft)": [3000, 6000, 9000, 12000, 3000, 6000, 9000, 6000, 3000, 6000, 6000, 12000, 3000, 3000, 6000],
```

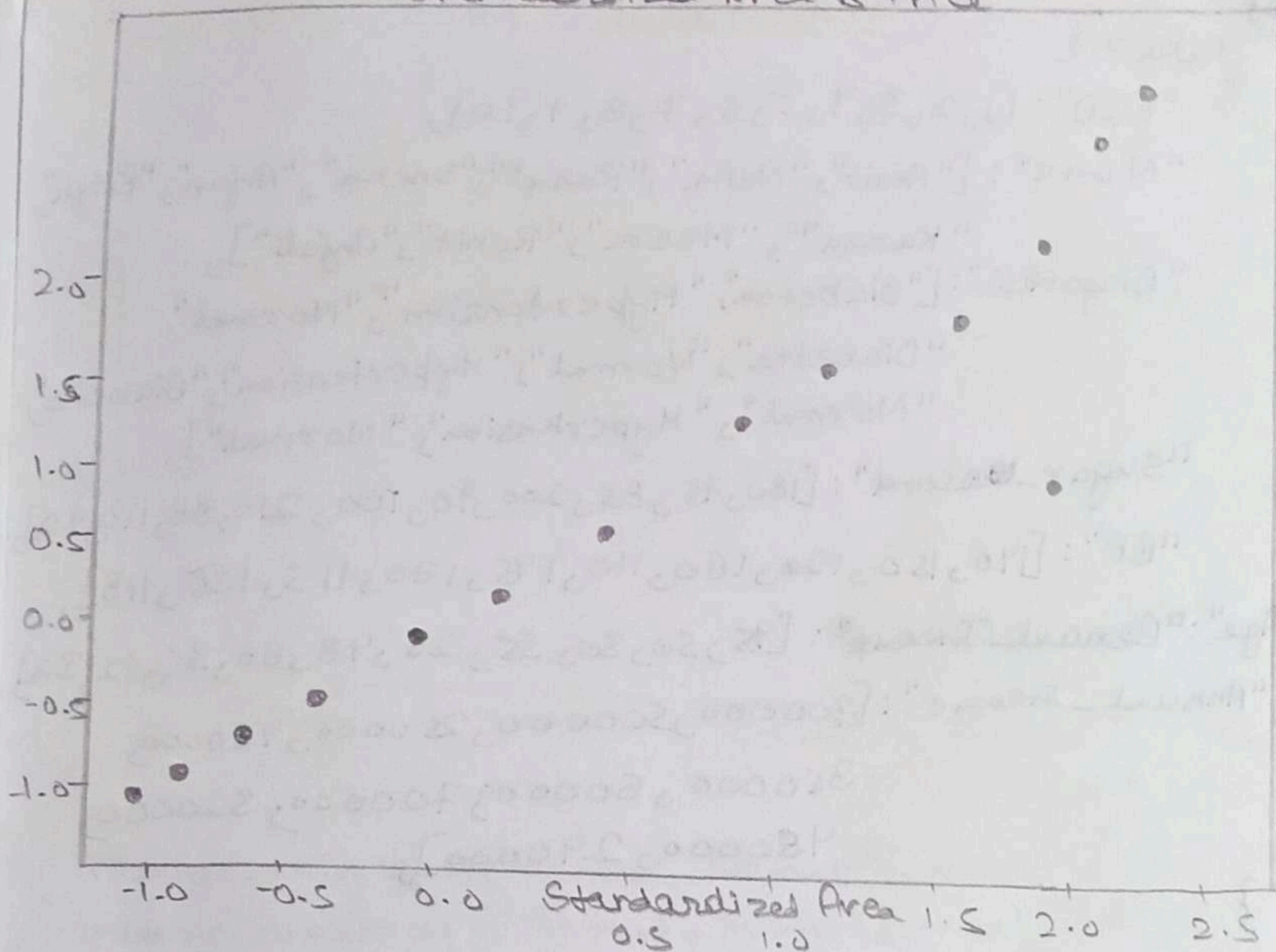
```
    "Locality of Residence": ["Rani Pokhari", "doiwala", "rampur", "dehri", "dehradun", "rishikesh", "chamoli", "almora", "sharanpur", "haridwar", "mohawala", "porandasaheb", "chamba", "haridwar", "rampur"]
```



```
print("DFS Traversal Order:")  
dfs(graph, root_node, visit_node)
```

```
graph = {  
    "A": ["B", "C"],  
    "B": ["D", "E"],  
    "C": ["F", "G"],  
    "D": ["H", "I"],  
    "E": ["J", "K"],  
    "F": ["L"],  
    "G": []  
}
```


Standardized Area VS Price



~~data =~~

~~1 2 3 4 5 6 7 8 9 10~~

~~11 12 13 14 15 16 17 18 19 20~~

output

std - area

std - Price

-1.069

-1.119

-0.066

0.0025

0.935

1.170

1.937

2.3185

-1.069

-1.119771

-0.066

0.0254

0.935

-1.1197

-0.066

0.0254

-1.069

0.0254

-0.0660

-1.1197

-0.0668

-1.1197

0.788

-1.069

0.025

-1.069

0.935

+1.937

0.0668

plt.figure()

plt.scatter(df2['std-area'], df2['std-Price'])

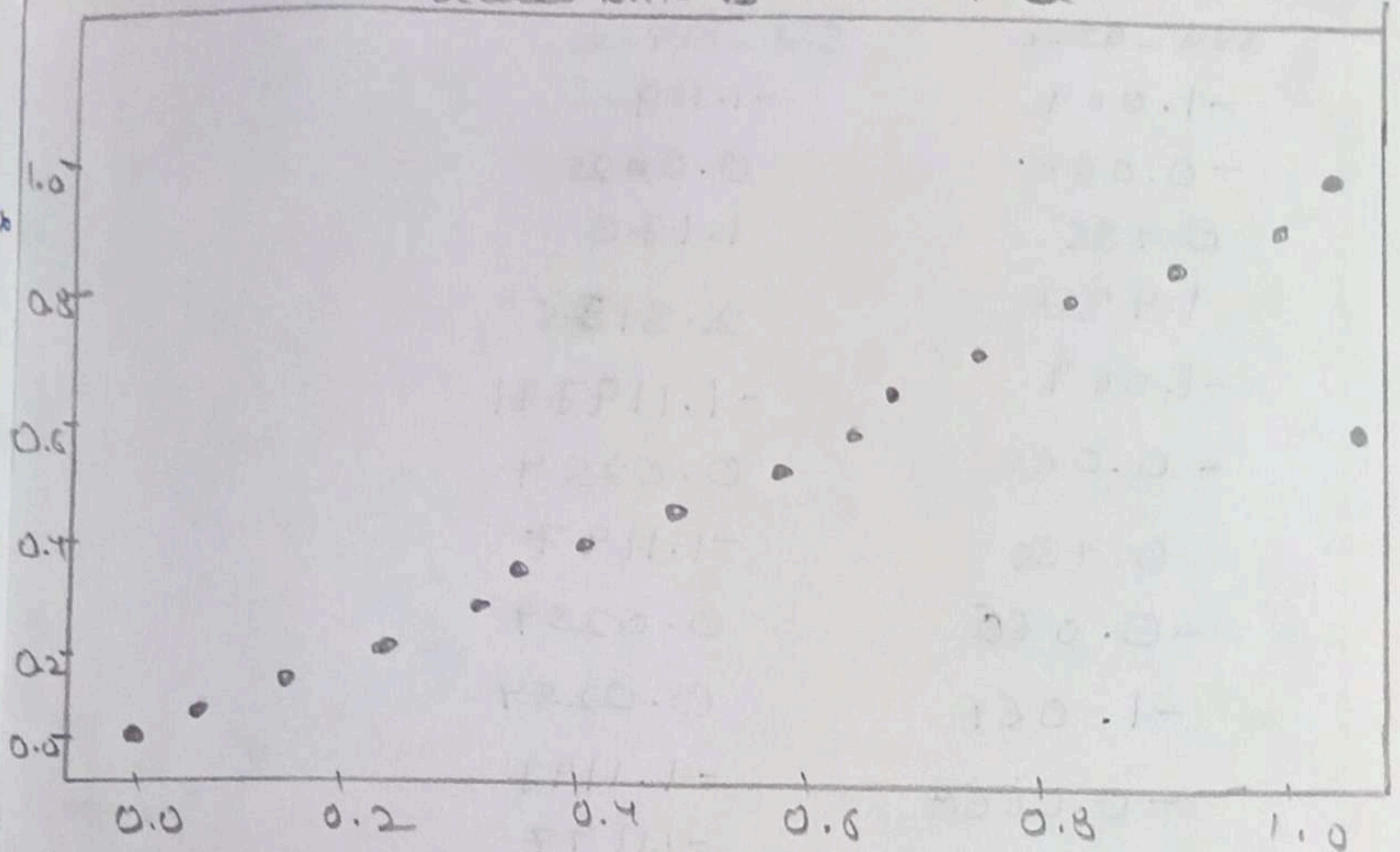
plt.xlabel("Standard-Area")

plt.ylabel("Standard Price")

plt.title("Standard Area VS Price")

plt.show()

Scaled BHK vs Scaled Price



from sklearn.preprocessing import StandardScaler
 scaler = StandardScaler()

df2[['std-area', 'std-price']] = scaler.fit_transform(df2[['Area-of-Residence (sq-ft)', 'Price-of-Residence']])

df2[['std-area', 'std-price']].head()

std-area	std-price
-1.069	-1.1197
-0.066	0.025479
+0.935	1.170669
1.937	2.315
-1.06	-1.119
-0.066	0.025

$$\begin{aligned} \text{Area_min} &= \text{df1}["\text{Area-of-Residence (sq-ft)}"].min() \\ \text{area_max} &= \text{df1}["\text{Area-of-Residence (sq-ft)}"].max() \\ \text{df1}["\text{Area-Normalized}"] &= (\text{df1}["\text{Area-of-Residence (sq-ft)}"] - \text{area_min}) / (\text{area_max} - \text{area_min}) \end{aligned}$$

$$\begin{aligned} \text{cost_min} &= \text{df1}["\text{Price-of-Residence}"].min() \\ \text{cost_max} &= \text{df1}["\text{Price-of-Residence}"].max() \\ \text{df1}["\text{Purchasing-Cost-Normalized}"] &= (\text{df1}["\text{Price-of-Residence}"] - \text{cost_min}) / (\text{cost_max} - \text{cost_min}) \end{aligned}$$

df1

Area-Normalized

0.000
0.333
0.666
1.000
0.000
0.333
0.666
0.333
0.000
0.333
0.333
0.000
0.000
0.555
0.333

Purchasing-Cost-Normalized

0.000
0.333
0.6667
1.000
0.000
0.333
0.666
0.333
0.000
0.333
0.333
0.000
0.000
1.000
0.333

27/3/26

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
```

```
df = pd.read_csv('student_data.csv')
```

df

SNO	Study-Hours	Sleep	Marks
1.	7	7	82.07
2.	4	6	72.8
3.	8	8	84.73
4.	5	7	65.98
5.	7	9	100.16
6.	3	6	40.07
7.	7	5	79.34
8.	8	5	89.45
9.	4	6	82.41
10.	8	9	88.96
11.	8	6	100.55
12.	8	8	74.99
13.	12	5	79.19
14.	2	7	69.77
15.	5	9	67.77
16.	1	5	66.46
17.	6	7	46.66
18.	9	7	40.94
19.	3	8	44.78
20.	7	9	75.48
21.	4	4	71.63
22.	3	9	

Annual-Income-Normalized

BP-Normalized

0.111111

0.500000

0.585556

0.66666

0.000000

0.166667

0.444444

0.833333

0.222222

0.000000

0.777778

0.583333

1.000000

1.000000

0.155556

0.083333

0.511111

0.250000

0.088889

0.183333

plt.scatter(df["Annual-Income-Normalized"],

df["BP-Normalized"])

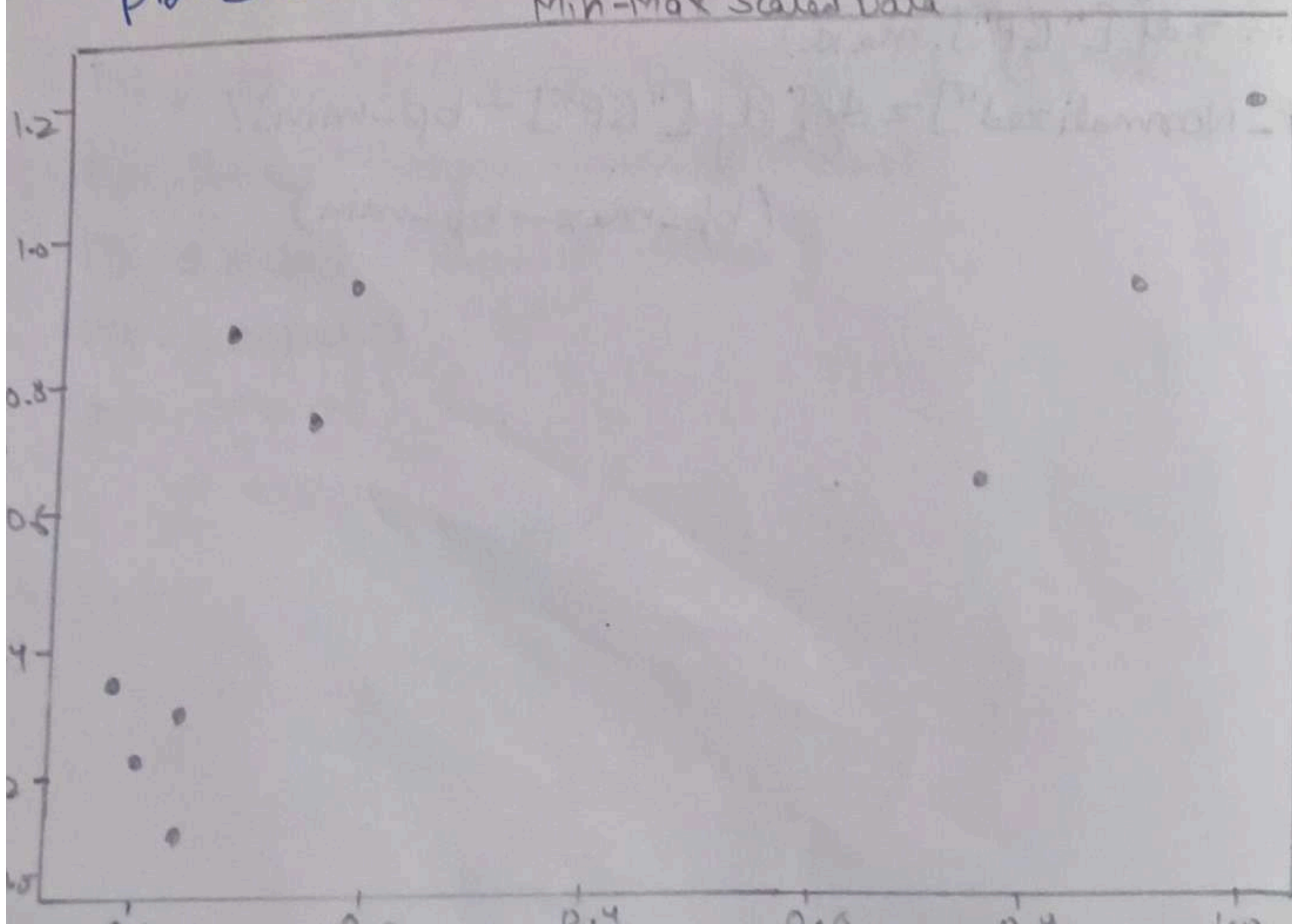
plt.title("After Normalization")

plt.xlabel("Normalized Annual Income")

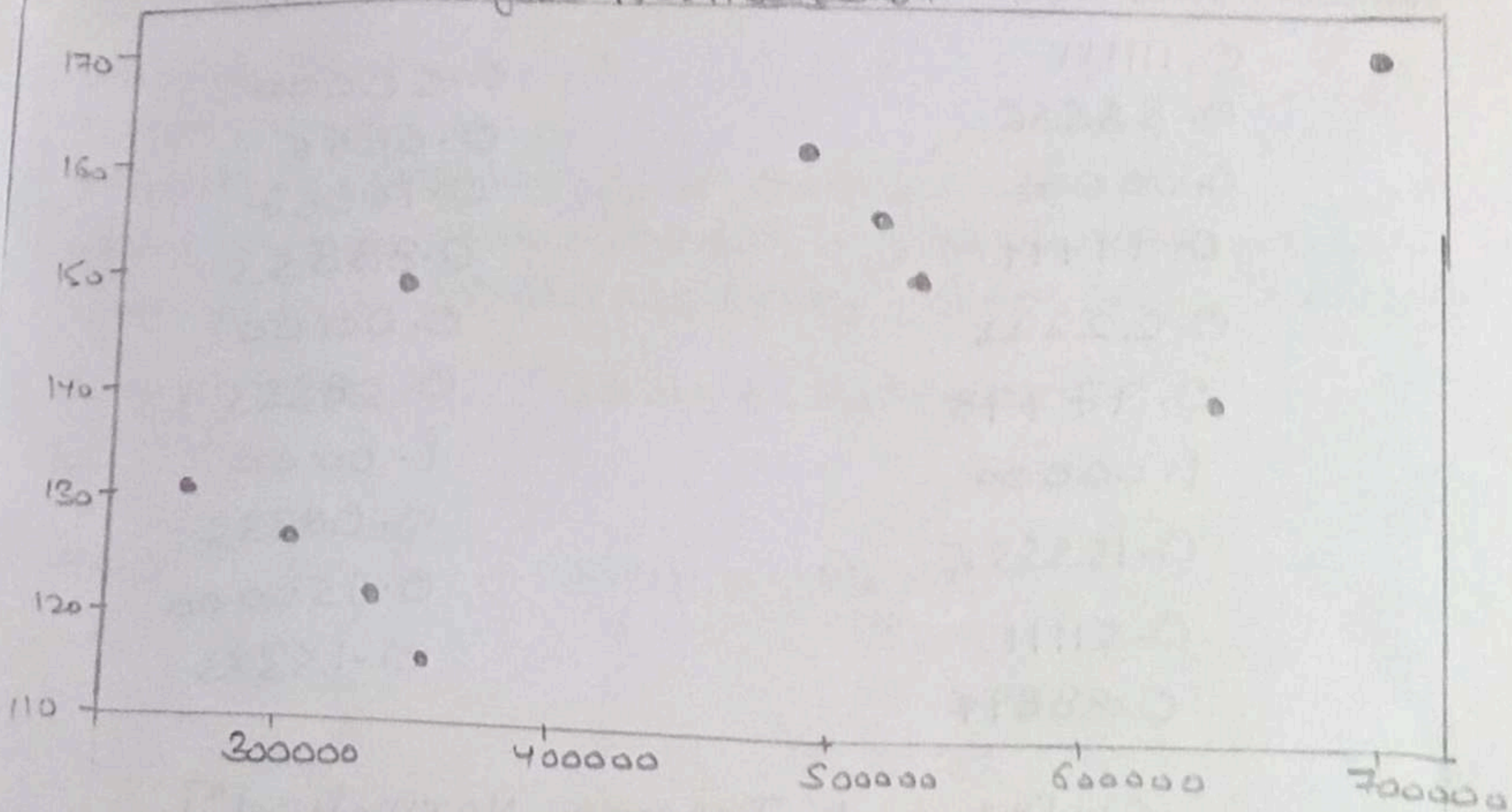
plt.ylabel("Normalized BP")

plt.show()

Min-Max Scaled Data



Before Normalization



$$\text{income_min} = df["\text{Annual_Income}"].min()$$

$$\text{income_max} = df["\text{Annual_Income}"].max()$$

$$df["\text{Annual_Income_Normalized}"] = (df["\text{Annual_Income}"] - \text{income_min}) / (\text{income_max} - \text{income_min})$$

$$\text{bp_min} = df["\text{BP}"].min()$$

$$\text{bp_max} = df["\text{BP}"].max()$$

$$df["\text{BP_Normalized}"] = (df["\text{BP}"] - \text{bp_min}) / (\text{bp_max} - \text{bp_min})$$

df

Ans 2)

data = {

"PID": [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],

"Name": ["Amit", "Neha", "Rahul", "Sneha", "Arjun", "Priya",
"Karen", "Meera", "Rohit", "Anjali"],

"Diagnosis": ["Diabetes", "Hypertension", "Normal",
"Diabetes", "Normal", "Hypertension", "Diabetes",
"Normal", "Hypertension", "Normal"],

"Sugar_Level": [180, 95, 85, 200, 90, 100, 210, 88, 110, 92],

"BP": [140, 150, 120, 160, 110, 145, 170, 115, 155, 118],

"Age": ~~Age~~ [45, 50, 30, 55, 28, 48, 60, 35, 32, 33],

"Annual_Income": [300000, 500000, 250000, 450000,
350000, 600000, 700000, 320000,
480000, 290000],

}

df = pd.DataFrame(data)

df

plt.scatter(df["Annual_Income"], df["BP"])

plt.title("Before Normalization")

plt.xlabel("Annual Income")

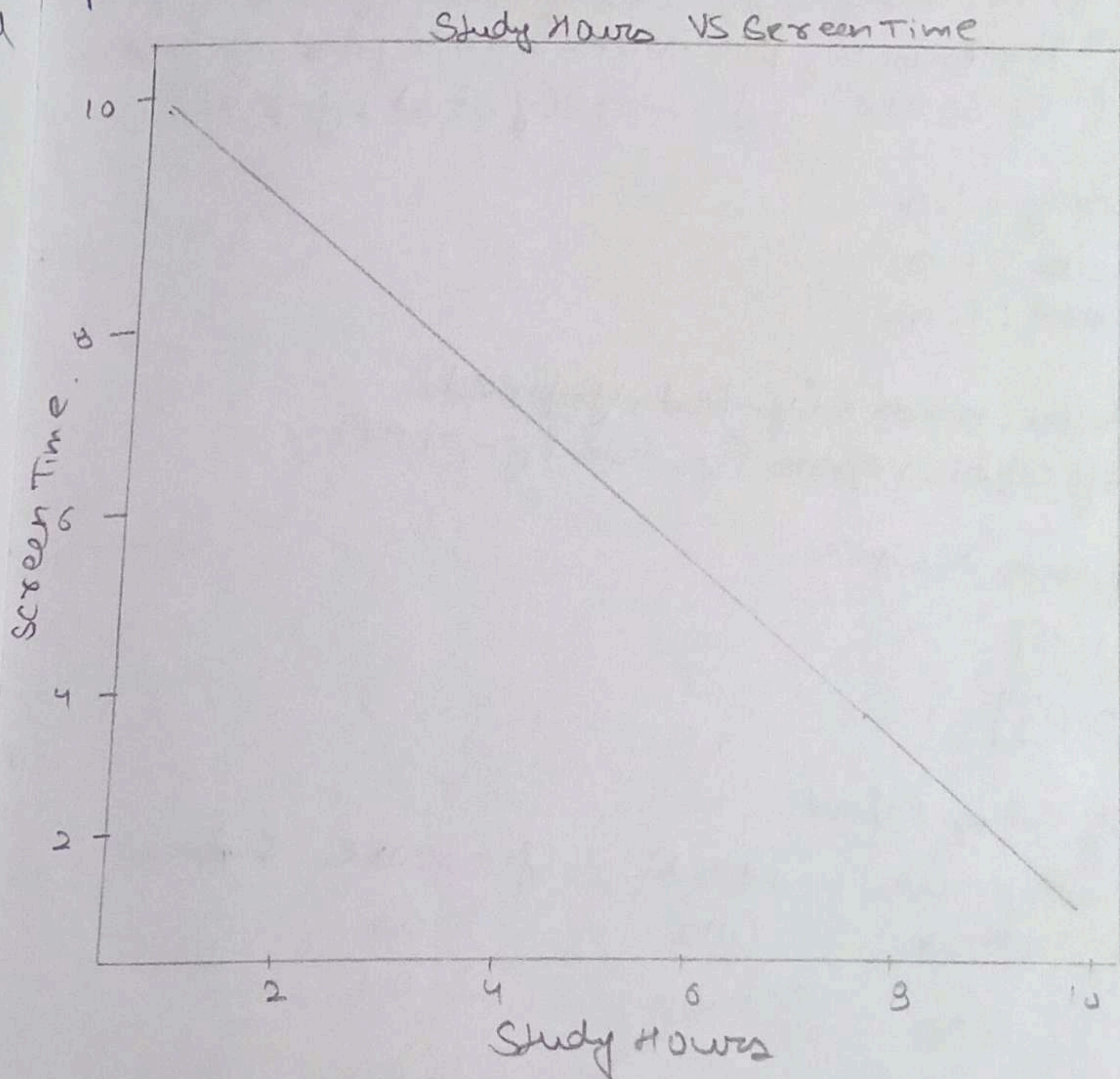
plt.ylabel("BP")

plt.show()


```

plt.plot(df["StudyHours"], df["ScreenTime"])
plt.xlabel("Study Hours")
plt.ylabel("Screen Time")
plt.title("StudyHours vs Screen Time")
plt.show()

```



```

selected_features = ["StudyHours", "ScreenTime"]
X = df[selected_features]
Y = df["Result"]

```

```

X_train, X_test, y_train, y_test = train_test_split
(X, y, test_size = 0.2, random_state = 42)

```

```

knn = KNeighborsClassifier(n_neighbors = 3)

```

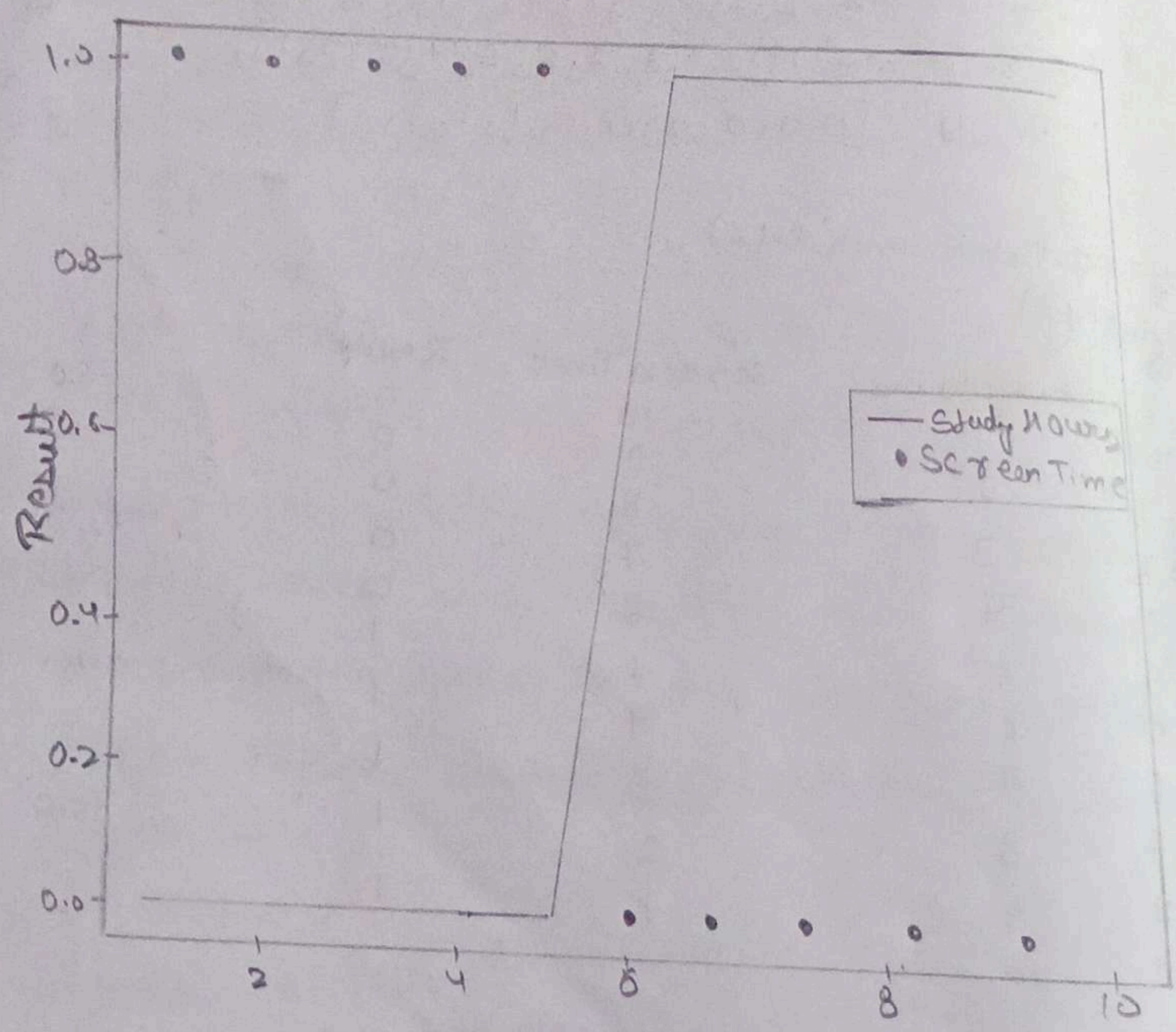
```

knn.fit(X_train, y_train)

```


~~plt.figure()~~
~~plt.plot(df["StudyHours"], df["Result"], color="red", label="StudyHours")~~
~~plt.scatter(df["ScreenTime"], df["Result"], label="ScreenTime")~~
~~plt.ylabel("Result")~~
~~plt.legend()~~
~~plt.show()~~

```
plt.figure()
plt.plot(df["StudyHours"], df["Result"], color="red", label="StudyHours")
plt.scatter(df["ScreenTime"], df["Result"], label="ScreenTime")
plt.ylabel("Result")
plt.legend()
plt.show()
```



plt.plot
 plt.xlabel
 plt.ylabel
 plt.colorbar
 plt.show

10
 ScreenTime
 8
 6
 4

KNN

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score,
precision_score, confusion_matrix, f1_score,
classification_report.
```

```
data = {
    "StudyHours": [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
    "ScreenTime": [10, 9, 8, 7, 6, 5, 4, 3, 2, 1],
    "Result": [0, 0, 0, 0, 0, 1, 1, 1, 1, 1]
}
```

```
df = pd.DataFrame(data)
print(df)
```

	StudyHours	ScreenTime	Result
0	1	10	0
1	2	9	0
2	3	8	0
3	4	7	0
4	5	6	0
5	6	5	1
6	7	4	1
7	8	3	1
8	9	2	1
9	10	1	1

```
print("\nShape:", df.shape)
```

```
Shape: (10, 3)
```


$x_{\text{train}}, x_{\text{test}}, y_{\text{train}}, y_{\text{test}} = \text{train_test_split}(x, y, \text{test_split}=0.2, \text{random_state}=2)$

`model = LinearRegression()`

`model.fit(x_train, y_train)`

`y_pred = model.predict(x_test)`

`print(model.intercept_)`

→ 34.75432

⊗

`print(model.coef_)`

→ [5.167. 2.0275]

`from sklearn.metrics import r2_score`

`r2 = r2_score(y_test, y_pred)`

`print("r-squared", r2)`

`n = x_test.shape[0]`

`p = x_test.shape[1]`

`adjusted_r2 = 1 - (1 - r2) * (n - 1) / (n - p - 1)`

`print("adjusted r2:" adjusted_r2)`

`from sklearn.metrics import mean_absolute_error`

`from sklearn.metrics import mean_squared_error`

`mae = mean_absolute_error(y_test, y_pred)`

`mse = mean_squared_error(y_test, y_pred)`

`print(mae)`

`print(mse)`

→ ~~0.78~~ 4181.194

→ ~~0.80~~ 33596915.85



Today

20:58



```
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Precision:", precision_score(y_test, y_pred))
print("F1 score:", f1_score(y_test, y_pred))
```

Accuracy : 1.0
 Precision : 1.0
 F1 Score : 1.0

Confusion_matrix(y_test, y_pred)
 Classification_report(y_test, y_pred)

Confusion Matrix

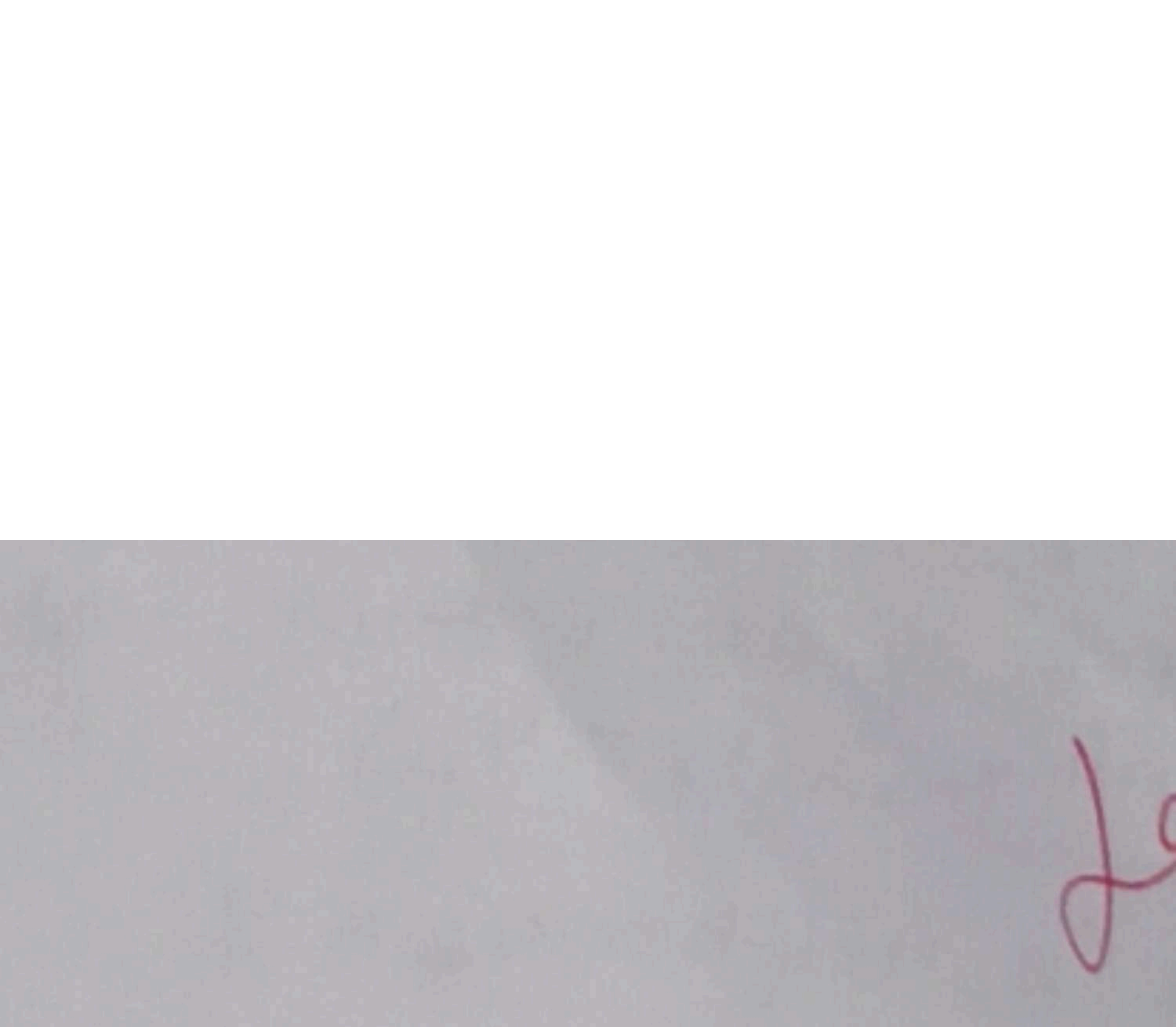
[[1 0]
 [0 1]]

Classification Report:

	Precision	recall	f1-Score	support
0	1.00	1.00	1.00	1
1	1.00	1.00	1.00	1
accuracy		1.00	1.00	2
macro avg	1.00		1.00	2
weighted avg	1.00	1.00	1.00	2

Late






```
y-pred = knn.predict(X-test)
print("Y predicted: ", y-pred)
→ Y predicted: [1.0]
```

```
print("Accuracy: ", accuracy_score(y-test, y-pred))
print("Precision: ", precision_score(y-test, y-pred))
print("F1 score: ", f1_score(y-test, y-pred))
```

Accuracy: 1.0

Precision: 1.0

F1 Score: 1.0

Confusion_matrix(y-test, y-pred)

Classification_report(y-test, y-pred)

Confusion Matrix

[[1 0]

[0 1]]

Classification Report:

	Precision	recall	f1-Score	support
0	1.00	1.00	1.00	1
1	1.00	1.00	1.00	1
accuracy		1.00	1.00	2
macro avg	1.00		1.00	2
weighted avg	1.00	1.00	1.00	2